

Lambda Calculus

Fondamento della programmazione funzionale

Le origini nella teoria della
calcolabilità

David Hilbert

Entscheidungsproblem

Il problema della decisione

È possibile definire una procedura, eseguibile del tutto meccanicamente, in grado di stabilire, per ogni formula espressa nel linguaggio formale della logica del primo ordine, se tale formula è o meno un teorema della logica del primo ordine?



Come si formalizza la nozione di procedura effettiva (algoritmo)? Diverse risposte



Alonzo Church: Lambda calculus

An unsolvable problem of elementary number theory, *Bulletin the American Mathematical Society*, May 1935

Come si formalizza la nozione di procedura effettiva (algoritmo)? Diverse risposte



Kurt Gödel: Recursive functions

Stephen Kleene, General recursive functions of natural numbers, *Bulletin the American Mathematical Society*, July 1935

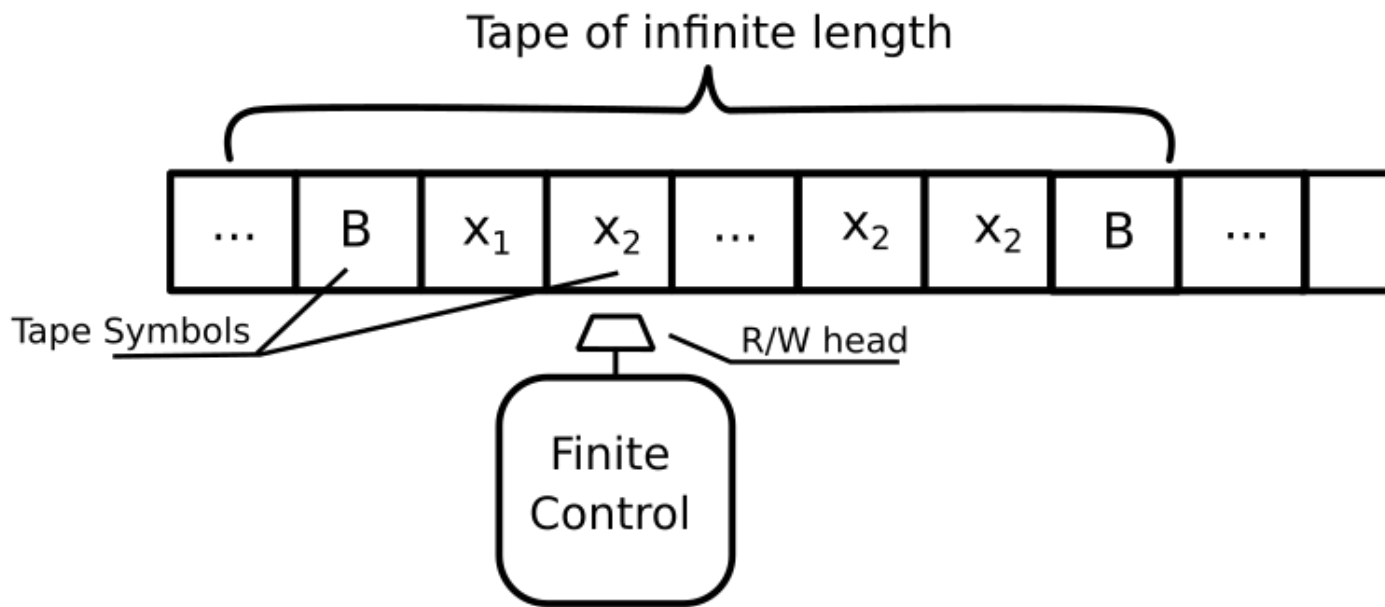
Come si formalizza la nozione di procedura effettiva (algoritmo)? Diverse risposte



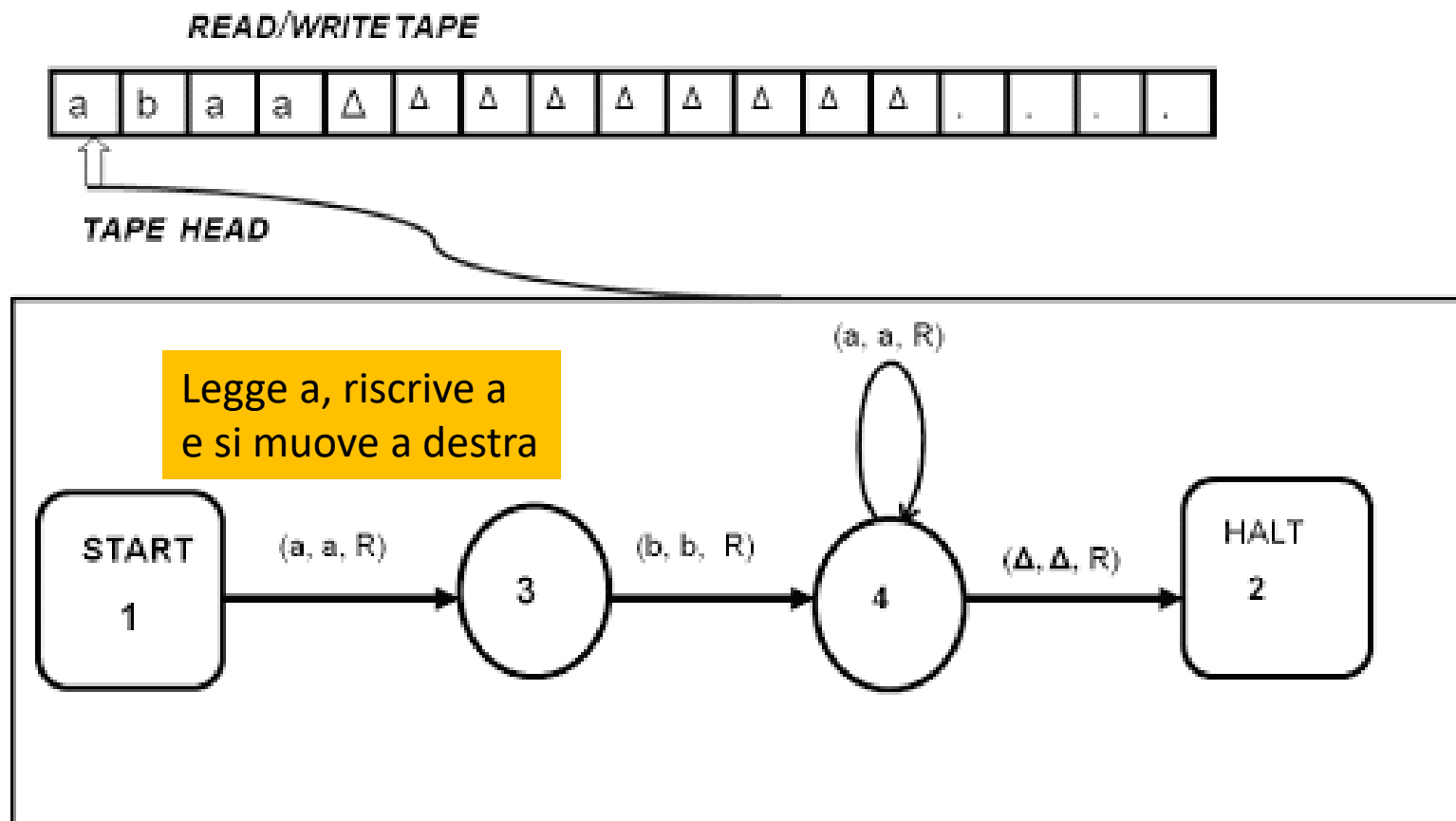
Alan M. Turing: Turing machines

On computable numbers, with an application to the *Entscheidungsproblem*, *Proceedings of the London Mathematical Society*, received 25 May 1936

Dalla Macchina di Turing...



- È una macchina ideale (non fisica) di calcolo che manipola i dati contenuti su un nastro di lunghezza potenzialmente infinita, secondo un insieme di regole ben definite
- È il modello astratto di una macchina in grado di eseguire algoritmi

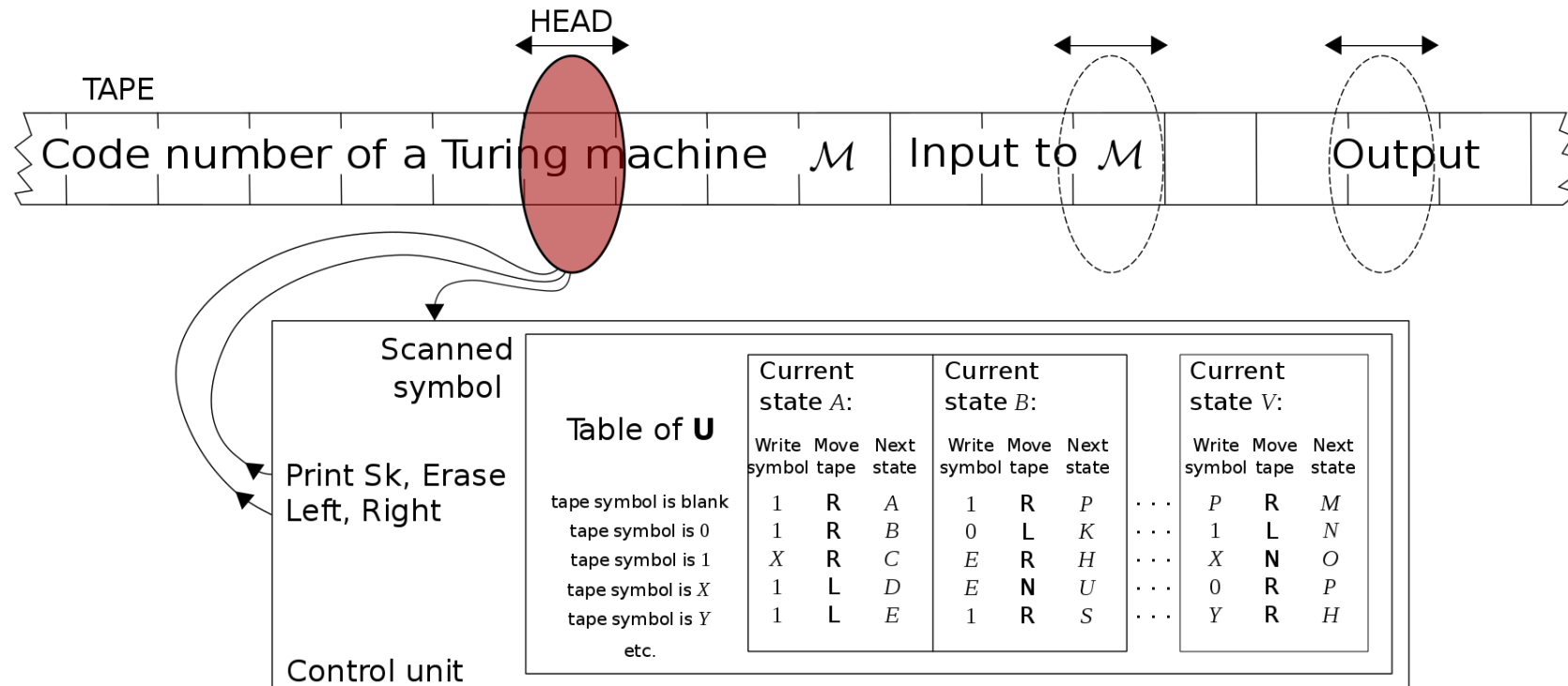


A Turing Machine for aba^*

...ai programmi memorizzati

- Se abbiamo una descrizione del funzionamento della macchina possiamo simularne il comportamento con carta e matita
- Questo processo è esso stesso un procedimento di calcolo
- Tra tutte le macchine di Turing ce ne è una che è in grado di simulare tutte le altre: è la Macchina di Turing Universale
- Con una descrizione e i dati la Macchina di Turing Universale eseguirà il calcolo per noi: è questo un modello teorico di computer

...ai programmi memorizzati



Macchina di Turing Universale

Macchina di Turing Universale (UTM)

La macchina di Turing universale (UTM) è una macchina di Turing capace di **simulare il comportamento di una qualunque macchina di Turing**

La UTM è stata definita da Turing nel suo fondamentale lavoro del 1936.

Gli ha consentito di dare una risposta negativa al "Entscheidungsproblem" (problema della decidibilità): esistono problemi che non possono essere risolti da nessuna macchina di calcolo

Tesi di Church-Turing

Se una **funzione** è **calcolabile** (tramite un qualunque formalismo: lambda calcolo, funzioni ricorsive, ecc...), allora esiste una macchina di Turing in grado di calcolarla

Questa tesi è indimostrabile (i formalismi sono potenzialmente infiniti), ma è universalmente accettata

Tesi di Church-Turing

- Modelli di calcolo meno potenti di una macchina di Turing:
 - **espressioni regolari (automi a stati finiti)**
 - **macchine che terminano sempre (funzioni totali)**
- Considerazione: non è noto un modello di calcolo più potente della macchina di Turing in termini computazionali
 - Quindi tutto ciò che non è calcolabile dalla TM non può essere calcolato da nessun altro formalismo **a noi noto**

Turing completeness

Un linguaggio di programmazione è detto **Turing completo** se è in grado di *calcolare qualsiasi funzione calcolabile da una macchina di Turing*

Come si fa a dimostrare che un linguaggio di programmazione è Turing completo?

- *Definendo una traduzione delle macchine di Turing in programmi scritti nel linguaggio: il programma simula il comportamento della macchina di Turing*

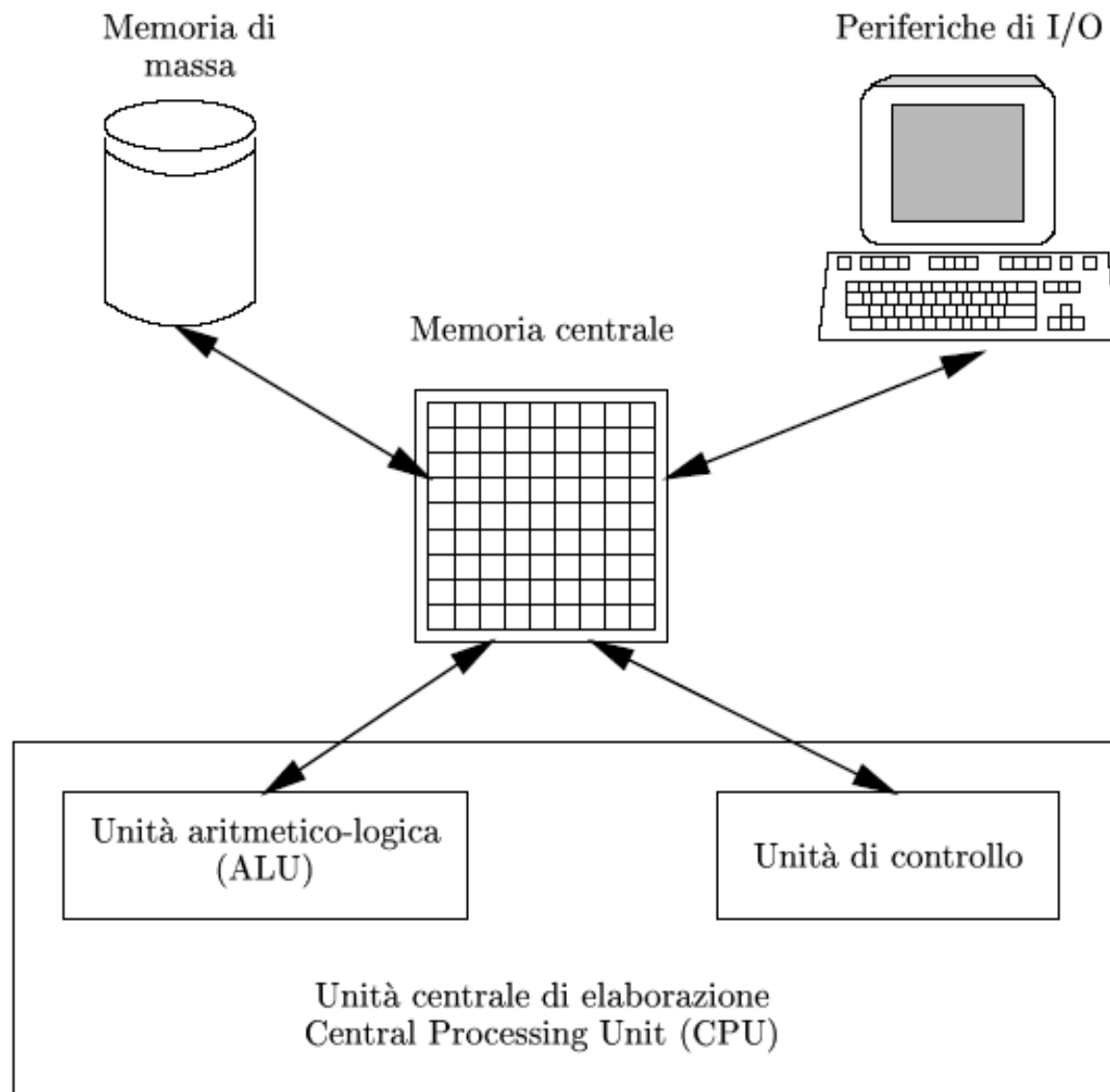
Dalle Macchine di Turing alla macchina di von Neumann

- Il modello di calcolo delle machine di Turing ha ispirato **John von Neumann** che ha definito la base della struttura dei computer attuali
- Si tratta di un modello concreto

L'**Architettura di von Neumann** ha due componenti principali

- **Memoria**, dove sono memorizzati i programmi e i dati
- **Unità centrale di elaborazione (CPU)**, che ha il compito di eseguire i programmi immagazzinati in memoria prelevando le istruzioni (e i dati relativi), interpretandole ed eseguendole una dopo l'altra
- La generalità è data dal programma memorizzato

Architettura di von Neumann



(Turing)-von Neumann programming languages

- Un linguaggio di programmazione nello stile (Turing-)von Neumann è un linguaggio di programmazione che prevede **astrazioni** di programmazione che riproducono ad alto livello la struttura della architettura di von Neumann
- **variabili** <astrazione> celle di memoria (MdT tape)
- **istruzioni di controllo** <astrazione> istruzioni di test&jump/if/cicli (MdT control)
- **assegnamento** <astrazione> modifica dello stato (fetching & storing instructions)

Turing-von Neumann Languages

1957 – FORTRAN

1959 – ALGOL

1962 – SIMULA

1972 – C

1979 – C++

1991 – Python

1995 – Java

Backus

- John Backus ha osservato che **la nozione di assegnamento** nei linguaggi di von Neumann divide la programmazione in due mondi.
- Il primo mondo è fatto di **espressioni**, uno spazio matematico ordinato con proprietà algebriche potenzialmente utili: la maggior parte dei calcoli avviene nel mondo delle espressioni
- Il secondo mondo è lo spazio delle **istruzioni**, uno spazio matematico disordinato con poche proprietà utili (la programmazione strutturata può essere vista come un'euristica limitata che cerca di porre un minimo di ordine a questo spazio)

Programmazione funzionale

- La **programmazione funzionale** è un paradigma di programmazione in cui il flusso di esecuzione del programma assume la forma di una serie di valutazioni di funzioni matematiche
- Il punto di forza principale di questo paradigma è la **mancaanza di effetti collaterali** (*side-effect*) delle funzioni, il che comporta una più facile verifica della correttezza (assenza di bug) del programma e la possibilità di una maggiore ottimizzazione dello stesso
- Il **Lambda calcolo** (Church 1935) può essere considerato il primo linguaggio di programmazione funzionale e può servire come banco di prova per studiare le caratteristiche dei linguaggi funzionali

Turing languages

1957 – FORTRAN

1959 – COBOL, ALGOL

1962 – SIMULA

1972 – C, Smalltalk

1979 – C++

1991 – Python

1995 – Java



Church languages

1959 – LISP

1966 – ISWIM

1972 – Prolog

1978 – ML

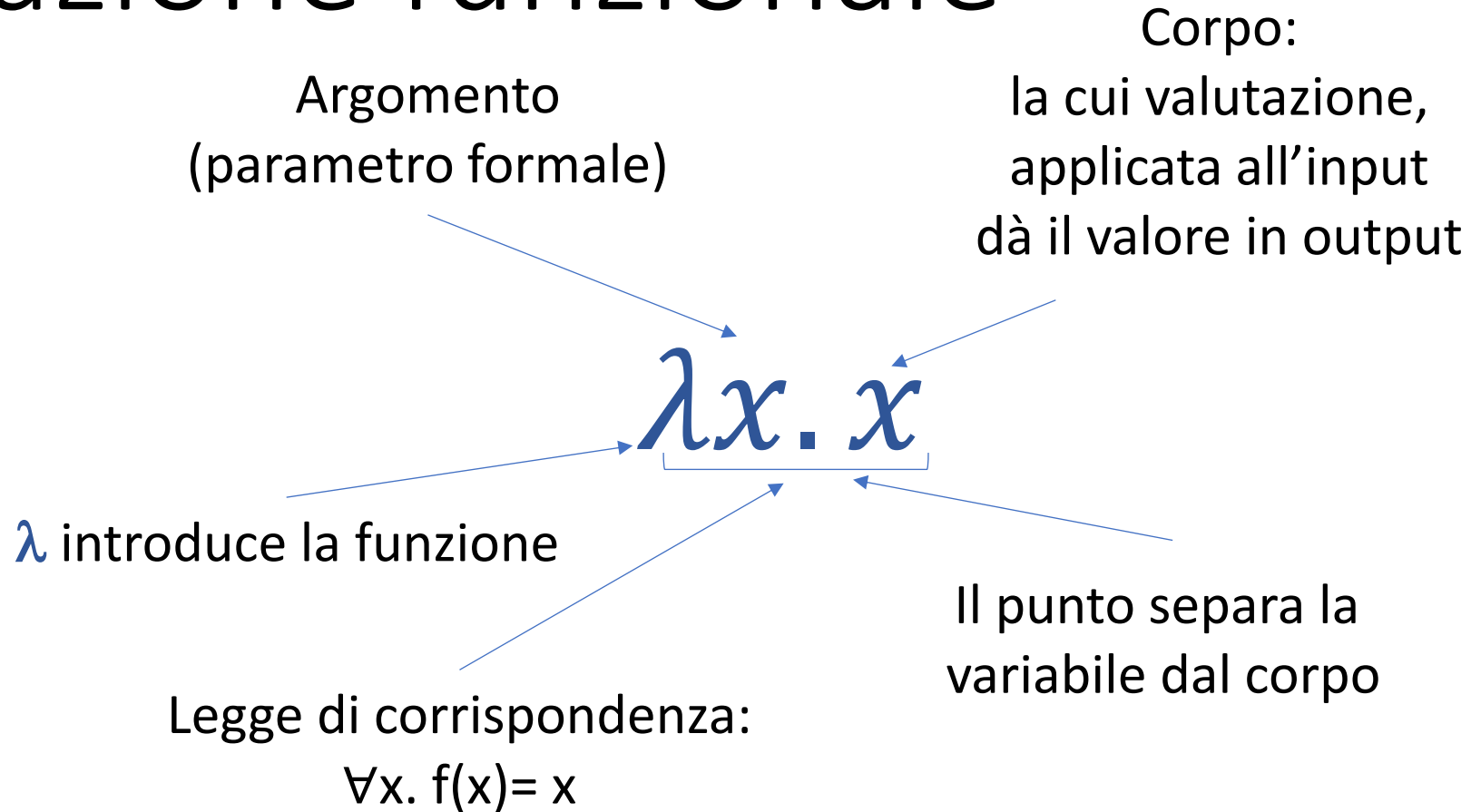
1990 – Haskell



Il lambda calcolo*

* Vedi anche dispensa

Astrazione funzionale



Questa funzione prende in ingresso x e restituisce x : è la funzione **identità**

Applicazione

Argomento
(parametro formale)

$$(\lambda x. x)y \longrightarrow y$$

y è il parametro attuale

Questa funzione applicata a y e restituisce y (è la funzione **identità**)

Sintassi

Si possono associare significati alle espressioni in un modo che corrisponde alle computazioni (programmi funzionali)

Un programma è una **espressione** (detta **lambda-espressione**)

Ci sono tre modi di formare termini (dato un insieme di variabili)

$e ::= x$

$| \lambda x.e$

$| e e$

variabile

astrazione funzionale (fun decl)

applicazione (fun call)

Niente altro!

La sintassi è terminata

Esempi lambda astrazioni

- $\lambda x.x$ è la funzione identità
- $\lambda y.(\lambda x.x)$ che fa?

Esempi lambda astrazioni

- $\lambda x.x$ è la funzione identità
- $\lambda y.(\lambda x.x)$ che fa? È la funzione che scarta il suo argomento y e restituisce la funzione identità
- $\lambda f.f(\lambda x.x)$ che fa?

Esempi lambda astrazioni

- $\lambda x.x$ è la funzione identità
- $\lambda y.(\lambda x.x)$ che fa? È la funzione che scarta il suo argomento y e restituisce la funzione identità
- $\lambda f.f(\lambda x.x)$ che fa? È la funzione che, data una funzione f , la invoca sulla funzione identità

Esempi applicazioni

- $(\lambda x.x)y$ restituisce y
- $(\lambda x.((\lambda y.y)x))z$ Che cosa restituisce?

Esempi applicazioni

- $(\lambda x.x)y$ restituisce y
- $(\lambda x.((\lambda y.y)x))z$ Che cosa restituisce? $(\lambda y.y)z$ e quindi z
- $(\lambda f.fz)(\lambda x.x)$ Che cosa restituisce?

Esempi applicazioni

- $(\lambda x.x)y$ restituisce y
- $(\lambda x.((\lambda y.y)x))z$ Che cosa restituisce? $(\lambda y.y)z$ e quindi z
- $(\lambda f.fz)(\lambda x.x)$ Che cosa restituisce? $(\lambda x.x)z$ e quindi z
- $(\lambda x.(xx))(\lambda y.y)$ Che cosa restituisce?

Esempi applicazioni

- $(\lambda x.x)y$ restituisce y
- $(\lambda x.((\lambda y.y)x))z$ Che cosa restituisce? $(\lambda y.y)z$ e quindi z
- $(\lambda f.fz)(\lambda x.x)$ Che cosa restituisce? $(\lambda x.x)z$ e quindi z
- $(\lambda x.(xx))(\lambda y.y)$ Che cosa restituisce? $(\lambda y.y)(\lambda y.y)$ e quindi $(\lambda y.y)$
- $((\lambda x.\lambda y.x+y)3)5$ Che cosa restituisce?

Esempi applicazioni

- $(\lambda x.x)y$ restituisce y
- $(\lambda x.((\lambda y.y)x))z$ Che cosa restituisce? $(\lambda y.y)z$ e quindi z
- $(\lambda f.fz)(\lambda x.x)$ Che cosa restituisce? $(\lambda x.x)z$ e quindi z
- $(\lambda x.(xx))(\lambda y.y)$ Che cosa restituisce? $(\lambda y.y)(\lambda y.y)$ e quindi $(\lambda y.y)$
- $((\lambda x.\lambda y.x+y)3)5$ Che cosa restituisce? $(\lambda y.3+y)5$ e quindi $3+5$ e quindi 8

Esempi applicazioni

- $(\lambda x.x)y$ restituisce y
- $(\lambda x.((\lambda y.y)x))z$ Che cosa restituisce? $(\lambda y.y)z$ e quindi z
- $(\lambda f.fz)(\lambda x.x)$ Che cosa restituisce? $(\lambda x.x)z$ e quindi z
- $(\lambda x.(xx))(\lambda y.y)$ Che cosa restituisce? $(\lambda y.y)(\lambda y.y)$ e quindi $(\lambda y.y)$
-  $((\lambda x.\lambda y.x+y)3)5$ Che cosa restituisce?  $(\lambda y.3+y)5$ e quindi $3+5$ e quindi 8 : notare che $(\lambda y.3+y)$ è una funzione che somma 3 al parametro che gli viene passato

Altri esempi

- $(\lambda x. \lambda y. xy)(\lambda x. x)z$ Che cosa restituisce? [la forma è $e_1 e_2 e_3$]

Altri esempi

- $(\lambda x. \lambda y. xy)(\lambda x. x)z$ Che cosa restituisce? [la forma è $e_1 e_2 e_3$]
restituisce $(\lambda y. (\lambda x. x)y)z$ e ancora

Altri esempi

- $(\lambda x. \lambda y. xy)(\lambda x. x)z$ Che cosa restituisce? [la forma è $e_1 e_2 e_3$]
restituisce $(\lambda y. (\lambda x. x)y)z$ e ancora $(\lambda x. x)z$ e infine z
- $(\lambda x. \lambda y. xy)zk$ Che cosa restituisce?

Altri esempi

- $(\lambda x. \lambda y. xy)(\lambda x. x)z$ Che cosa restituisce? [la forma è $e_1 e_2 e_3$] restituisce $(\lambda y. (\lambda x. x)y)z$ e ancora $(\lambda x. x)z$ e infine z
- $(\lambda x. \lambda y. xy)zk$ Che cosa restituisce? $(\lambda y. zy)k$ e infine zk

Esempi a cui fare attenzione 1

$(\lambda x.x)((\lambda y.y)z)$ Che cosa restituisce?

[la forma è $e_1 (e_{21}e_{22})$]

Esempi a cui fare attenzione 1

$(\lambda x.x)((\lambda y.y)z)$ Che cosa restituisce?

[la forma è $e_1 (e_{21} e_{22})$]

- $(\lambda y.y)z$ se scelgo il redex più esterno: $(\lambda x.x)((\lambda y.y)z)$

Esempi a cui fare attenzione 1

$(\lambda x.x)((\lambda y.y)z)$ Che cosa restituisce?

[la forma è $e_1 (e_{21} e_{22})$]

- $(\lambda y.y)z$ se scelgo il redex più esterno: $(\lambda x.x)((\lambda y.y)z)$
- $(\lambda x.x)z$ se scelgo quello più interno $((\lambda y.y)z)$

Esempi a cui fare attenzione 1

$(\lambda x.x)((\lambda y.y)z)$ Che cosa restituisce?

[la forma è $e_1 (e_{21} e_{22})$]

- $(\lambda y.y)z$ se scelgo il redex più esterno: $(\lambda x.x)((\lambda y.y)z)$
- $(\lambda x.x)z$ se scelgo quello più interno $((\lambda y.y)z)$

Notare che in ogni caso alla fine ottengo z

Esempi a cui fare attenzione 2

$(\lambda x. \lambda y. xy)y$ Che cosa restituisce?

Esempi a cui fare attenzione 2

$(\lambda x. \lambda y. x y) y$ Che cosa restituisce?

- Se procedessi come prima otterrei $\lambda y. y y$: è corretto?

Esempi a cui fare attenzione 2

$(\lambda x. \lambda y. xy)y$ Che cosa restituisce?

- Se procedessi come prima otterrei $\lambda y. yy$: è corretto?

Se avessi $(\lambda x. \lambda z. xz)y$ che è del tutto equivalente, cosa otterrei?

- Otterrei $\lambda z. yz$

Non sono risultati equivalenti, le due funzioni si comportano in modo diverso

C'è un **conflitto di nomi**

Lambda espressioni: funzioni anonime

- L'espressione $\lambda x.e$ è una funzione **anonima** (funzione a cui non viene associato alcun nome)
 - Il nome x è la dichiarazione del parametro della funzione anonima

Funzioni anonime nei linguaggi

- Javascript
 - `function(a){ return a + 1; }` oppure `a => a+1`
- OCaml
 - `fun x -> x+1`
- Java (si chiamano anche Lambda)
 - `(int x) -> x + 1`

Lambda espressioni: applicazione

- In $e_1 e_2$ l'espressione della funzione e_1 è applicata all'espressione dell'argomento e_2
- L'**applicazione** corrisponde alla **chiamata di funzione** in Javascript
 - l'espressione e_1 definisce la funzione
 - l'espressione e_2 definisce il parametro attuale
- Il calcolo λ è molto generale e permette alle definizioni di funzione di apparire direttamente nelle chiamate di funzione

Ad esempio:

$$(\lambda x. (\lambda y. xy))(\lambda z. z)$$

e_1

e_2

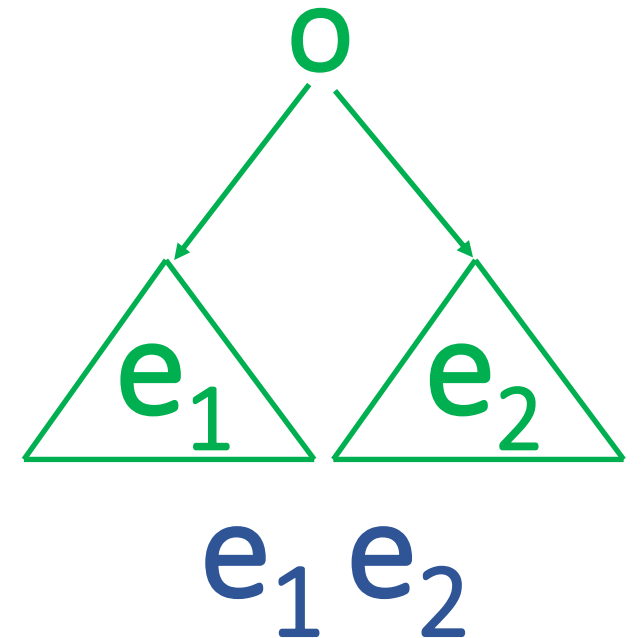
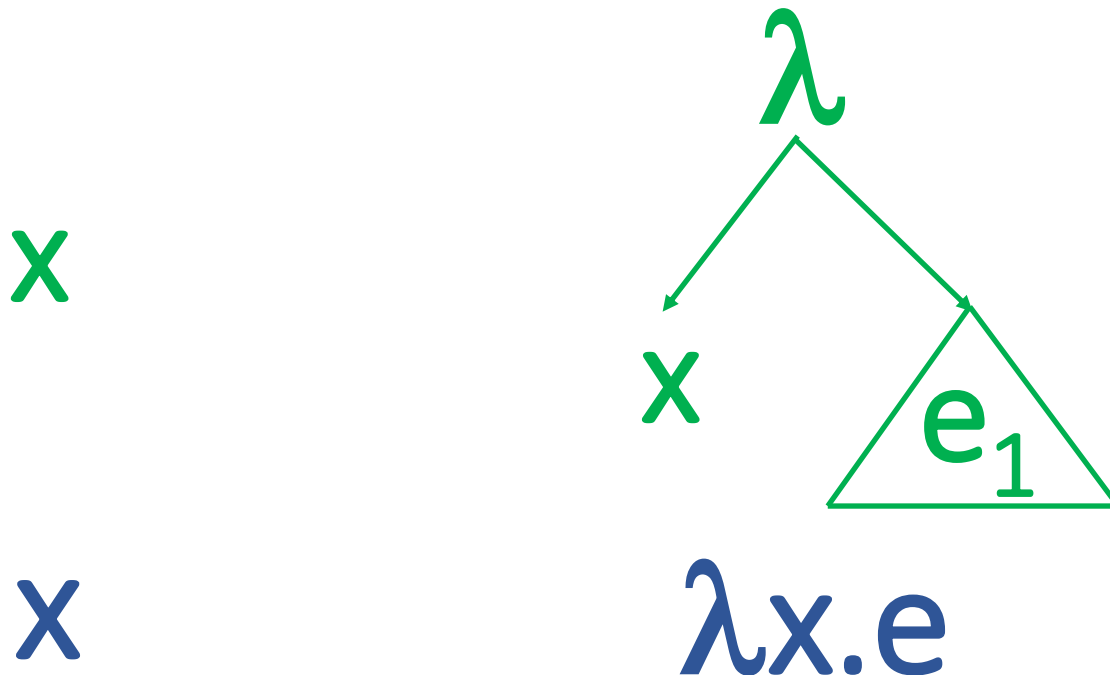
Convenzioni sintattiche (importanti)

- Costruttore lambda: la portata (lo scope) del lambda si estende il più a destra possibile
 - $\lambda x. \lambda y. x \ y$ è la stessa cosa di $\lambda x. (\lambda y. (x \ y))$
- L'applicazione funzionale associa a sinistra
 - $e_1 \ e_2 \ e_3$ è la stessa cosa di $(e_1 \ e_2) \ e_3$

Esercizio: Quali parentesi sono sottintese in $\lambda x. x \ \lambda y. xyz$?

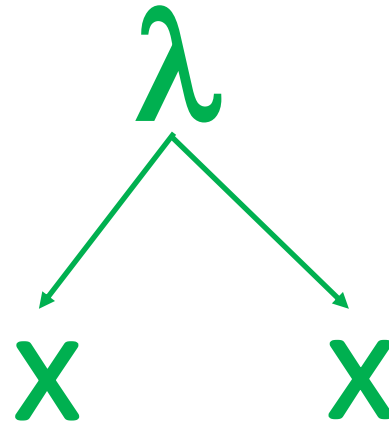
Alberi per λ espressioni

- Possiamo dare una rappresentazione ad albero delle espressioni del Lambda calcolo, seguendo l'annidamento (completo) delle parentesi
- Utile per capire quali passi di valutazione sono appropriati



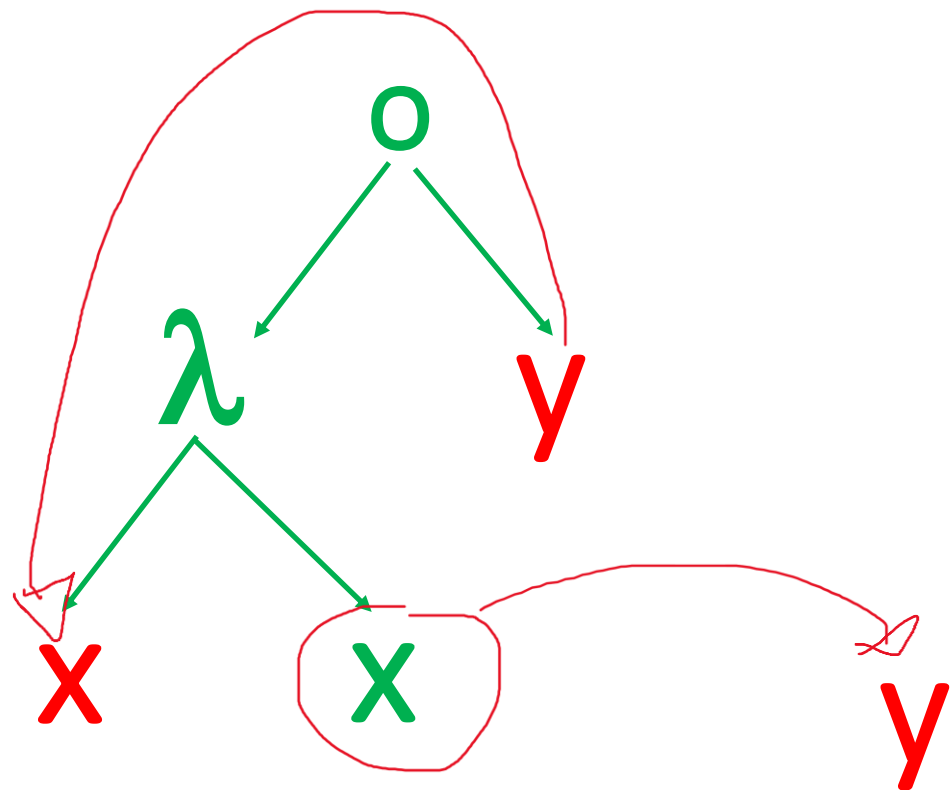
Alberi: astrazione

$\lambda x.x$



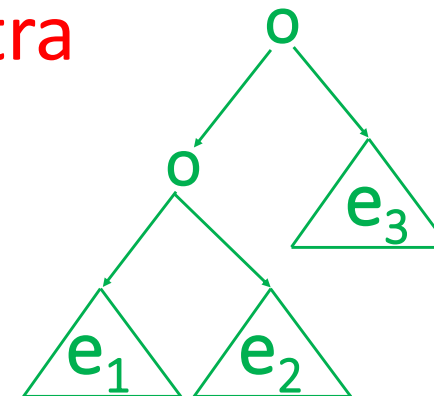
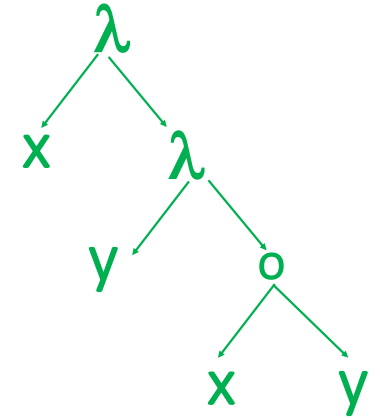
Alberi: applicazione

$(\lambda x.x)y \rightarrow y$



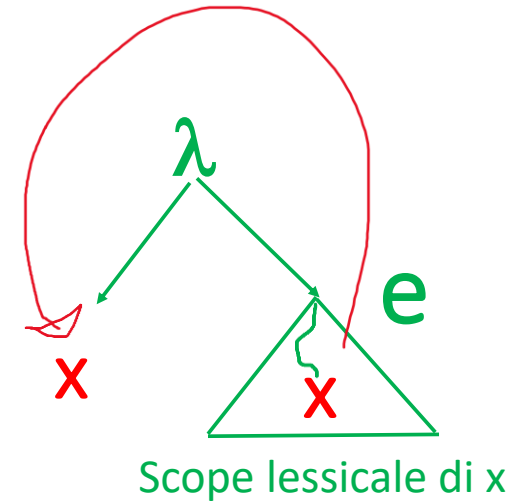
Convenzioni sintattiche (importanti) di nuovo

- Costruttore lambda: la portata (lo scope) del lambda si estende il più a destra possibile
 - $\lambda x. \lambda y. x y$ è la stessa cosa di $\lambda x. (\lambda y. (x y))$
- L'applicazione funzionale associa a sinistra
 - $e_1 e_2 e_3$ è la stessa cosa di $(e_1 e_2) e_3$



λ : operatore di binding

- Il λ nella lambda astrazione $\lambda x.e$ è un **operatore che lega** le variabili (il suo **scope** è e), come $\forall$ in $\forall x.P$ della logica del prim'ordine (dove P può contenere x)
- x è una variabile fittizia (un *placeholder*) che potrebbe essere rimpiazzata con qualsiasi altra variabile *fresca*
- Una *variabile fresca* è un nome nuovo: un nome che non compare da nessuna parte in nessuna delle espressioni che stiamo trattando



Il λ binding è l'unico modo di associare valori a variabili

Variabile libere e legate

- Le variabili in una lambda espressione e che sono introdotte (dichiarate) in un qualche λ (che sono cioè nello scope di λ) sono **legate** da quel λ : lo scope delle variabili è il termine e
- Tutte le altre variabili che non sono associate a una dichiarazione con un qualche λ sono invece **libere**

Esempio



$\lambda x.xy$

- x è una variabile **legata**, perché si trova all'interno del corpo dell'espressione λx
- y è una variabile **libera**, perché non si trova all'interno di un'espressione λy

Variabile libere e legate: esempi

- $\lambda x.x$
- $\lambda x.\lambda y.(xyz)$
- $(\lambda f.fx)y$
- $(\lambda f.fx)f$

Variabile libere e legate: esempi

- $\lambda x.x$  x è legata
- $\lambda x.\lambda y.(xyz)$  x e y sono legate z è libera
- $(\lambda f.fx)y$  f è legata, x e y sono libere
- $(\lambda f.fx)f$  la prima f è legata, x e la seconda f sono libere (attenzione allo **scope** di λ !)

Variabile libere e legate: attenzione!

Gli **scope** possono essere **annidati** e quindi una variabile può avere un significato diverso nei diversi contesti, creando un po' di confusione ovvero un **conflitto di nomi**

$\lambda x.x \ (\lambda x.x)x$

Variabili libere: definizione formale

L'insieme delle **variabili libere**, **FV (Free Variables)**, di una lambda espressione è definito induttivamente dalle equazioni

$$FV(x) = \{x\}$$

$$FV(e_1 e_2) = FV(e_1) \cup FV(e_2)$$

$$FV(\lambda x.e) = FV(e) \setminus \{x\}$$

- Un termine si dice **chiuso** se $FV(e) = \emptyset$
- I termini chiusi si chiamano **combinatori**: il più semplice è l'identità $\lambda x.x$
- Una variabile che non è libera è **legata** (**bound**)

Esercizio: $FV((\lambda x.\lambda y.xy)(\lambda z.z)k) = ?$

Alpha equivalenza

Le espressioni

Le variabili **a** e **b** non hanno un significato,:
contano per il ruolo all'interno dell'espressione

$\lambda a.ac$ e $\lambda b.bc$

sono dette **α -equivalenti**

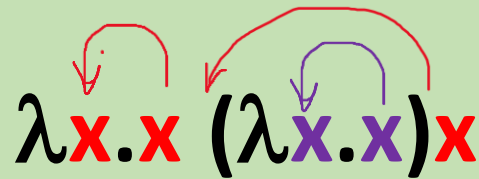
Sostituire una variabile legata con un'altra *opportuna variabile fresca* costituisce una trasformazione sintattica di programmi che solitamente (in matematica) viene chiamata **α -conversione**

function(a){ return a + 1; }
è **α -equivalente** a
function(b){ return b + 1; }

Espressioni **α -equivalenti** rappresentano lo stesso programma

Alpha-conversione

Consente di cambiare i nomi delle variabili legate e di passare, ad esempio, da $\lambda x.x$ a $\lambda y.y$ e anche risolvere il problema di *variable shadowing* visto prima

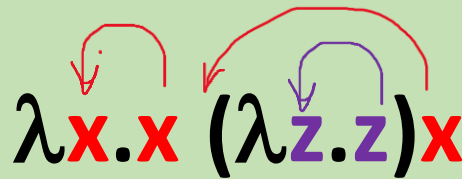


The diagram shows the lambda expression $\lambda x.x (\lambda x.x)x$ on a light green background. The first x in the body is red. The second lambda abstraction has a red x and a purple x . The body of the second lambda has a purple x and a red x . A red arrow points from the red x in the first lambda to the red x in the body of the second lambda. A purple arrow points from the purple x in the second lambda to the purple x in its body. This illustrates how alpha-conversion can be used to rename variables to avoid shadowing.

$$\lambda x.x (\lambda x.x)x$$

Alpha-conversione

Consente di cambiare i nomi delle variabili legate e di passare, ad esempio, da $\lambda x.x$ a $\lambda y.y$ e anche risolvere il problema di *variable shadowing* visto prima



The diagram shows the lambda expression $\lambda x.x (\lambda z.z)x$. The first lambda abstraction is $\lambda x.x$ with red text. The second is $(\lambda z.z)x$ with purple text for the inner lambda and red text for the argument x . A red curved arrow points from the x in the first lambda to the x in the second lambda's argument, indicating a binding relationship. Another red curved arrow points from the z in the second lambda to its body z . A purple curved arrow points from the z in the second lambda to its body z , indicating a binding relationship.

$$\lambda x.x (\lambda z.z)x$$

I due termini sono equivalenti a meno di α -conversione

La nozione di esecuzione

Cosa significa *eseguire*
(*valutare*) una lambda
espressione?

La valutazione, *eval*, di
lambda espressioni
comporta solamente la
chiamata di funzioni

Valutazione dell'applicazione di funzione $((\lambda x.e_1) e_2)$

$\text{eval}((\lambda x.e_1) e_2)$

- Rimpiazzare ogni occorrenza di x in e_1 con e_2
- Valutare e restituire il termine che ne risulta

Intuizione (informatica)

La **chiamata** della funzione anonima $\lambda x.e_1$ con parametro attuale e_2 corrisponde a eseguire il corpo e_1 della funzione dopo la trasformazione sintattica nella quale ogni occorrenza del **parametro formale** x è sostituita con l'espressione **parametro attuale** e_2

La **notazione per la sostituzione** è

$$e_1\{x:=e_2\}$$

descrive la lambda espressione e_1 in cui ogni
occorrenza libera della variabile x è sostituita
dalla lambda espressione e_2

Esempio: $xz \{x:= \lambda y.y\} = (\lambda y.y)z$

Una **notazione alternativa** è

$$e_1\{e_2/x\}$$

Sostituzione

La **notazione per la sostituzione** è

$$e_1\{x:=e_2\}$$

descrive la lambda espressione e_1 in cui ogni
occorrenza libera della variabile x è sostituita
dalla lambda espressione e_2

Esempio: $xz \{x:= \lambda y.y\} = (\lambda y.y)z$

Una **notazione alternativa** è

$$e_1\{e_2/x\}$$

Problema

E se e_2 contiene una variabile libera y legata in e_1 ?

C'è il rischio che con la sostituzione y venga
catturata dal legatore λ di e_1

Legare una variabile libera non è una bella idea!

Sostituzione

La **notazione per la sostituzione** è

$$e_1\{x:=e_2\}$$

descrive la lambda espressione e_1 in cui ogni occorrenza libera della variabile x è sostituita dalla lambda espressione e_2

Esempio: $xz \{x:= \lambda y.y\} = (\lambda y.y)z$

Una **notazione alternativa** è

$$e_1\{e_2/x\}$$

Soluzione

Si rinominano le variabili (con l' **α -conversione**) per evitare ambiguità, introducendo una variabile nuova e non utilizzata. Questo processo serve a garantire che, durante la sostituzione, non si leghi accidentalmente una variabile libera, evitando così il problema di *cattura delle variabili*

Sostituzione

Capture-avoiding substitution: esempio
(usiamo operazioni aritmetiche, per semplicità)


$$(\lambda \mathbf{x} . (\mathbf{x} * \mathbf{y})) \{ \mathbf{y} := (\mathbf{x} + \mathbf{x}) \}$$

--- α -conversione, z fresco


$$(\lambda \mathbf{z} . (\mathbf{z} * \mathbf{y})) \{ \mathbf{y} := (\mathbf{x} + \mathbf{x}) \} = (\lambda \mathbf{z} . (\mathbf{z} * (\mathbf{x} + \mathbf{x})))$$

Capture-avoiding substitution: definizione

- $x\{x:=e\} \equiv e,$
- $y\{x:=e\} \equiv y$ se $x \neq y$
- $(e_1 e_2) \{x:=e\} \equiv (e_1 \{x:=e\})(e_2 \{x:=e\})$
- $(\lambda y.e_1)\{x:=e\} \equiv \lambda y.(e_1 \{x:=e\})$ se $y \neq x$ e $y \notin FV(e)$
- $(\lambda y.e_1)\{x:=e\} \equiv \lambda z.((e_1 \{y:=z\})\{x:=e\})$ se $y \neq x$ e $y \in FV(e)$
e z fresca
- $(\lambda x.e_1)\{x:=e\} \equiv \lambda x.e_1$

Se il parametro formale x coincide con la variabile da sostituire, la sostituzione **non** viene applicata (la x è già legata)

Se $y \in FV(e)$, allora la sostituzione $\{x:=e\}$ crea conflitti di nome: quindi prima di attuarla occorre **α -convertire** e_1 sostituendo y con una variabile fresca z , ovvero applicare la sostituzione $\{y:=z\}$

Capture-avoiding substitution: esempio

$$(\lambda x. \lambda y. ((\lambda z. z) x)) \mathbf{y} \stackrel{?}{\rightarrow} (\lambda y. ((\lambda z. z) x)) \{x := \mathbf{y}\}$$


Sono due y diversi: $\mathbf{y} \in FV(e) = y$

Capture-avoiding substitution: esempio

$(\lambda y. e_1)\{x:=e\} \equiv \lambda z. ((e_1 \{y:=z\})\{x:=e\})$ se $y \neq x$ e $y \in FV(e)$ e z fresca

$$(\lambda x. \lambda y. ((\lambda z. z)x))y \stackrel{?}{\rightarrow} (\lambda y. ((\lambda z. z)x)\{x:=y\})$$


Questo passo non va bene: sono due y diversi: $y \in FV(e) = y$
Quindi procedo come segue

$$\lambda y. ((\lambda z. z)x)y \stackrel{ok}{\rightarrow} ((\lambda y. ((\lambda z. z)x)y)\{y:=k\})\{x:=y\}$$

k è fresca

Capture-avoiding substitution: esempio

$(\lambda y. e_1)\{x:=e\} \equiv \lambda z. ((e_1 \{y:=z\})\{x:=e\})$ se $y \neq x$ e $y \in FV(e)$ e z fresca

$$(\lambda x. \lambda y. ((\lambda z. z)x))y \stackrel{?}{\rightarrow} (\lambda y. ((\lambda z. z)x)\{x:=y\})$$


Questo passo non va bene: sono due y diversi: $y \in FV(e) = y$
Quindi procedo come segue

$$\lambda y. ((\lambda z. z)x)y \stackrel{ok}{\rightarrow} ((\lambda y. ((\lambda z. z)x)y)\{y:=k\})\{x:=y\} =$$

k è fresca

$$((\lambda k. ((\lambda z. z)x)\{x:=y\}))$$

Capture-avoiding substitution: esempio

$(\lambda y. e_1)\{x:=e\} \equiv \lambda z. ((e_1 \{y:=z\})\{x:=e\})$ se $y \neq x$ e $y \in FV(e)$ e z fresca

$$(\lambda x. \lambda y. ((\lambda z. z)x))y \stackrel{?}{\rightarrow} (\lambda y. ((\lambda z. z)x)\{x:=y\})$$


Questo passo non va bene: sono due y diversi: $y \in FV(e) = y$
Quindi procedo come segue

$$\lambda y. ((\lambda z. z)x))y \stackrel{ok}{\rightarrow} ((\lambda y. ((\lambda z. z)x))y)\{y:=k\}\{x:=y\} =$$

k è fresca

$$((\lambda k. ((\lambda z. z)x))\{x:=y\}) = (\lambda k. ((\lambda z. z)y))$$

Capture-avoiding substitution: esercizi

- $((\lambda x. yx)w)\{x := \lambda k. kx\}$
- $((\lambda x. yx)w)\{y := \lambda k. kx\}$
- $((\lambda x. yx)w)\{w := \lambda k. kx\}$

Beta reduction (o valutazione)

La regola di riduzione β (**β -reduction**) è la regola fondamentale che definisce il lambda calcolo come modello di calcolo universale

$$(\lambda x. e_1)e_2 \rightarrow e_1\{x := e_2\}$$

- La regola di β -riduzione cattura precisamente la nozione di applicazione funzionale: applica una funzione ai suoi argomenti: $(\lambda x.x)3 \rightarrow 3$
- Il risultato è un'istanza del corpo dell'astrazione in cui le occorrenze dei parametri formali vengono sostituite dalle copie dell'argomento
- Un **redex**, o espressione riducibile, è una sottoespressione di un'espressione λ in cui una λ può essere applicata a un argomento: **$(\lambda x.x)3$**

Esempio

Parametro formale

Parametro attuale

$(\lambda x. (\lambda z. (x z))) y$ 

???

Esempio

Parametro formale

Parametro attuale

$$\left(\lambda \textcolor{red}{x}. (\lambda z. (\textcolor{red}{x} z)) \right) \textcolor{blue}{y} \quad \longrightarrow$$

???

Beta Reduction

$$(\lambda \textcolor{red}{x}. e_1) \textcolor{blue}{e}_2 \rightarrow e_1 \{ \textcolor{red}{x} := e_2 \}$$

$$e_1 = \lambda z. (x z) \quad e_2 = y$$

Istanziamo la regola

Esempio

Parametro formale

Parametro attuale

$$\left(\lambda \textcolor{red}{x}. \boxed{\lambda z. (\textcolor{red}{x} z)} \right) y \quad \longrightarrow$$

$$\boxed{\lambda z. (\textcolor{red}{x} z)}$$

Beta Reduction

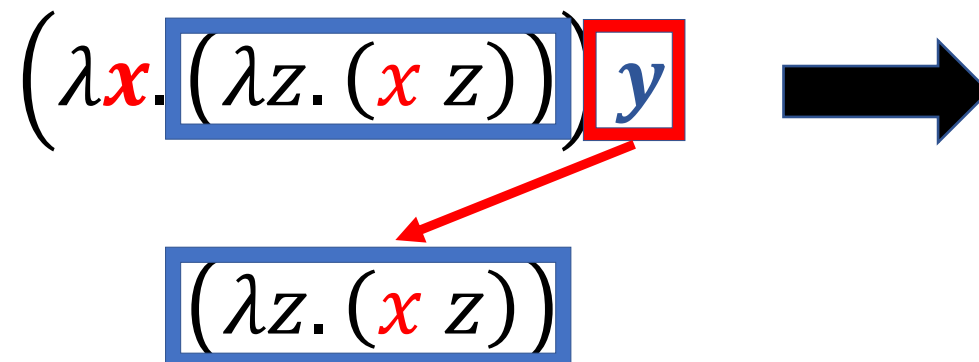
$$(\lambda \textcolor{red}{x}. \boxed{e_1}) e_2 \rightarrow \boxed{e_1} \{ \textcolor{red}{x} := e_2 \}$$

$$e_1 = \boxed{\lambda z. (x z)} \quad e_2 = y$$

Esempio

Parametro formale

Parametro attuale



Beta Reduction

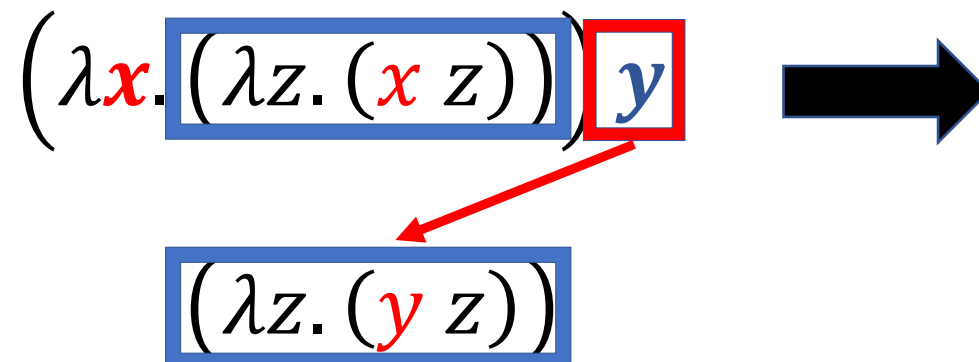
$$(\lambda x. e_1) e_2 \rightarrow e_1 \{x := e_2\}$$

$$e_1 = \lambda z. (x z) \quad e_2 = y$$

Esempio

Parametro formale

Parametro attuale



Beta Reduction

$$(\lambda x. e_1) e_2 \rightarrow e_1 \{x := e_2\}$$

$$e_1 = \lambda z. (x z) \quad e_2 = y$$

Esempio

Parametro formale

Parametro attuale

$$\left(\lambda \textcolor{red}{x}. (\lambda z. (\textcolor{red}{x} z)) \right) \textcolor{blue}{y} \quad \longrightarrow$$

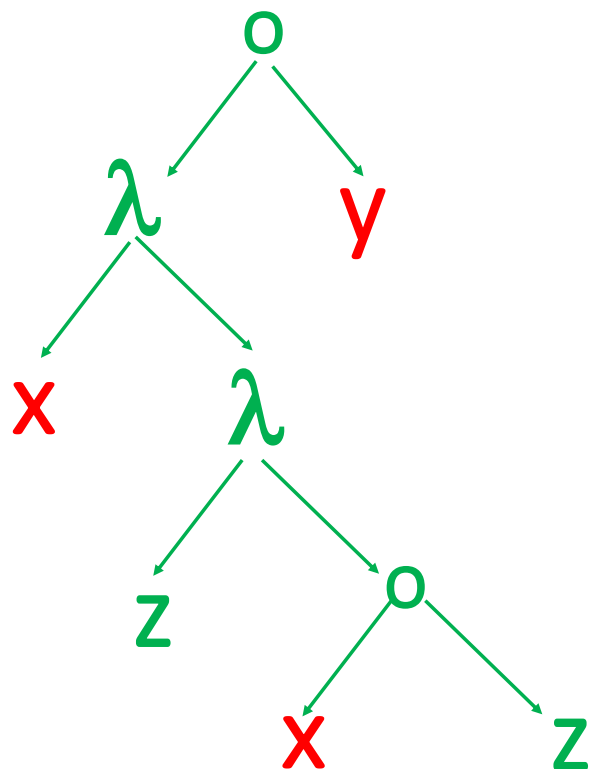
$$(\lambda z. (\textcolor{red}{y} z))$$

Beta Reduction

$$(\lambda \textcolor{red}{x}. e_1) \textcolor{blue}{e}_2 \rightarrow e_1 \{ \textcolor{red}{x} := e_2 \}$$

$$e_1 = \lambda z. (x z) \quad e_2 = y$$

Esempio

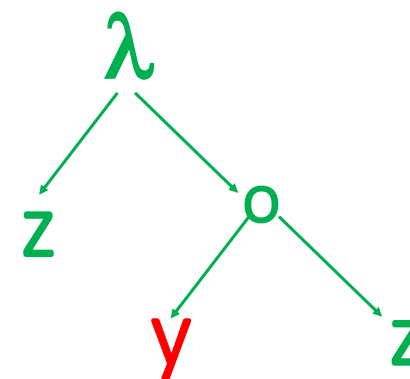


Parametro formale

Parametro attuale

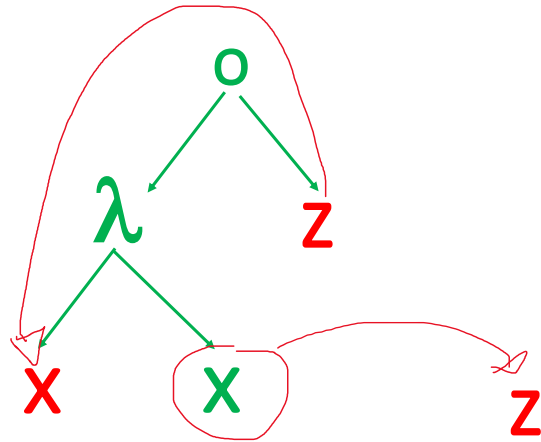
$$(\lambda \textcolor{red}{x}. (\lambda z. (\textcolor{red}{x} z))) \textcolor{blue}{y}$$

Red arrows indicate the mapping from the lambda expression to the AST: from λ to the first λ node, from $\textcolor{red}{x}$ to the x node, from λz to the inner λ node, from $\textcolor{red}{x}$ to the inner x node, and from $\textcolor{blue}{y}$ to the y node.

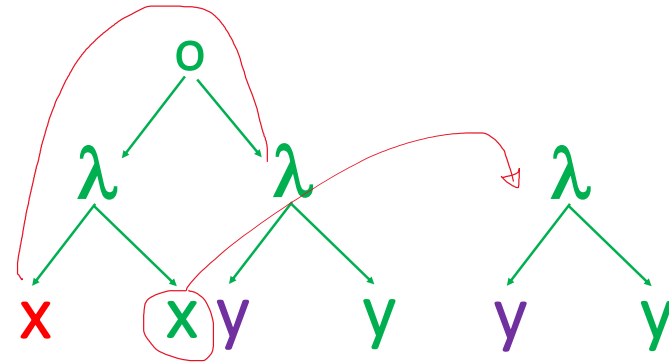


Esempi: funzione identità

$$(\lambda x. x) z \rightarrow z$$



$$(\lambda x. x) (\lambda y. y) \rightarrow (\lambda y. y)$$



Ordine superiore

Il lambda calcolo permette di scrivere in modo naturale funzioni che possono prendere altre funzioni come parametri e/o restituire funzioni come risultato

Esempi

- Una funzione che prende come argomento una funzione z e la applica a y

$$(\lambda x. x \ y)z \rightarrow zy$$

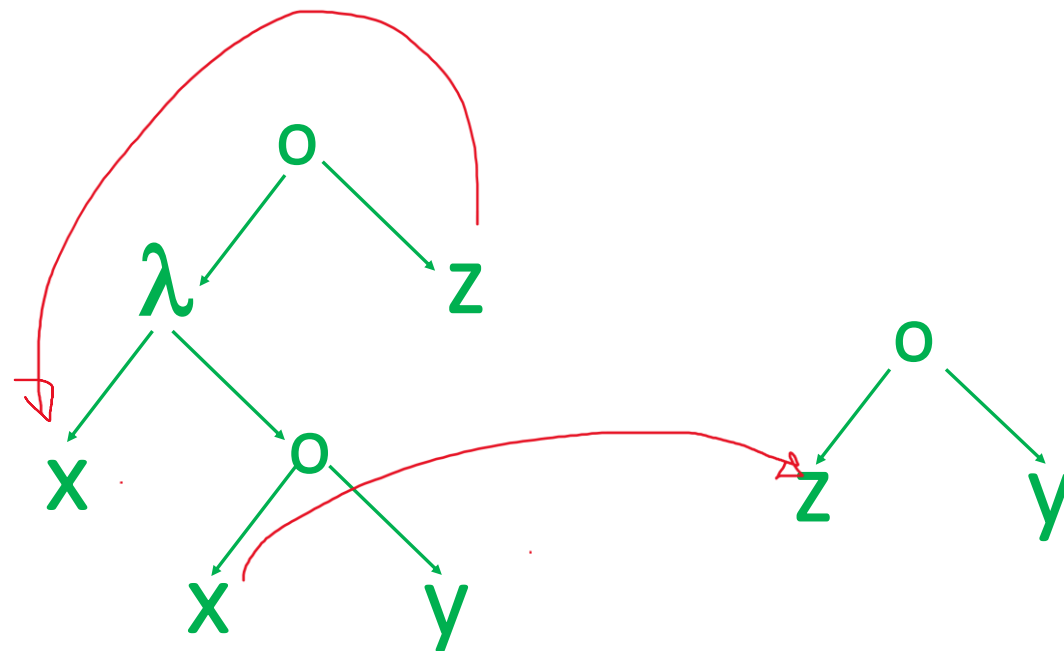
- Una funzione il cui parametro attuale è la funzione identità

$$(\lambda x. x \ y)(\lambda z. z) \rightarrow (\lambda z. z)y \rightarrow y$$

Esempi

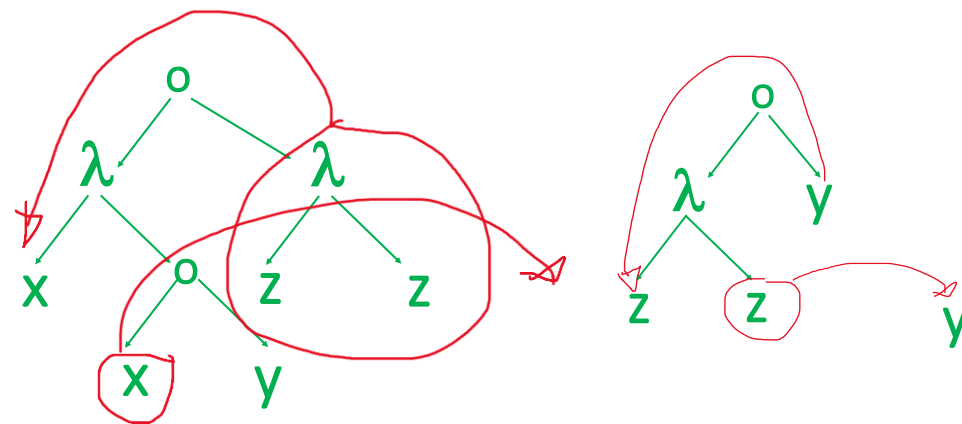
- Una funzione che prende come argomento una funzione z e la applica a y

$$(\lambda x. x \ y)z \rightarrow zy$$



- Una funzione il cui parametro attuale è la funzione identità

$$(\lambda x. x \ y)(\lambda z. z) \rightarrow (\lambda z. z)y \rightarrow y$$





Valori calcolati

- La valutazione procede scegliendo un'espressione riducibile e riducendola
- Quando una lambda espressione non può essere ridotta ulteriormente diciamo che è in **forma normale beta**
 - Una lambda espressione è in **forma normale beta** quando non è più riscrivibile per mezzo della regola di beta riduzione
- Dato che la beta riduzione rappresenta un passo di calcolo: la sua chiusura transitiva ne rappresenta una qualsiasi computazione
- Una lambda espressione ridotta in forma normale rappresenta pertanto il risultato finale, cioè **il valore calcolato**, della computazione

Esempi

- $\lambda x. x$ è un valore (le funzioni sono valori)
- $\lambda t. \lambda f. t$ è un valore

Beta riduzione, chiusura transitiva

- $e_1 \rightarrow e_2$ se e_2 in si ottiene da e_1 con **un passo** di riduzione
- $e_1 \Rightarrow e_2$ se e_2 in si ottiene da e_1 con **zero o più passi** di riduzione

Con $\Rightarrow$ indichiamo cioè la chiusura riflessiva e transitiva di $\rightarrow$

Questo equivale a dire che la relazione $\Rightarrow$ contiene $\rightarrow$ e a una regola di transitività alla definizione di $\Rightarrow$.

La lambda espressione e_1 è β -riducibile alla lambda espressione e_2 quando vale che $e_1 \Rightarrow e_2$

Beta equivalenza

- La relazione di riduzione $\Rightarrow$ fornisce le basi per poter definire una nozione di equivalenza tra lambda espressioni

Due lambda espressioni e_1 , e_2 si dicono **beta equivalenti**, $e_1 \equiv_\beta e_2$ se

- *Sono identiche (modulo α -conversione)*
- *$e_1 \Rightarrow e_2$ oppure $e_2 \Rightarrow e_1$*
- *$e_1 \Rightarrow e$ e anche $e_2 \Rightarrow e$*

Esempio

$(\lambda. x)z$ è **beta equivalente** a $(\lambda x. \lambda y. x)zw$ dato che entrambe si riducono alla stessa lambda espressione z

Beta equivalenza

- La relazione di riduzione $\Rightarrow$ fornisce le basi per poter definire una nozione di equivalenza tra lambda espressioni

Due lambda espressioni e_1 , e_2 si dicono **beta equivalenti**, $e_1 \equiv_\beta e_2$ se

- Sono identiche (modulo α -conversione)*
- $e_1 \Rightarrow e_2$ oppure $e_2 \Rightarrow e_1$*
- $e_1 \Rightarrow e$ e anche $e_2 \Rightarrow e$*

Notare che $(\lambda y. z)$ è come $f(y) = z$: restituisce sempre z , qualunque input riceva

Esempio

$(\lambda. x)z$ è **beta equivalente** a $(\lambda x. \lambda y. x)zw$ dato che entrambe si riducono alla stessa lambda espressione z

$(\lambda y. z)w \rightarrow z$

Beta equivalenza

Intuizione:

due lambda espressioni sono **beta equivalenti** quando sono indistinguibili dal punto di vista computazionale, calcolano cioè gli stessi risultati

Confluenza e non terminazione

Esempi a cui fare attenzione 1 (già visto)

$(\lambda x.x)((\lambda y.y)z)$ Che cosa restituisce?

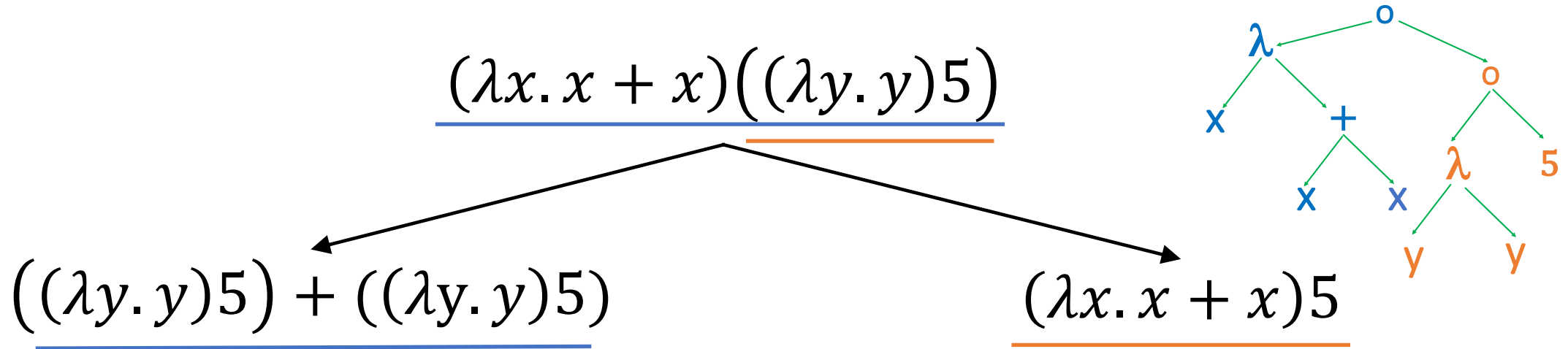
[la forma è $e_1 (e_{21} e_{22})$]

- $(\lambda y.y)z$ se scelgo il redex più esterno: $(\lambda x.x)((\lambda y.y)z)$
- $(\lambda x.x)z$ se scelgo quello più interno $(\lambda x.x)((\lambda y.y)z)$

In ogni caso alla fine ottengo z

Ordine di riduzione: esempio

In caso di più redex, esiste più di un ordine di valutazione, ovvero più di una scelta per applicare le regola di β -riduzione



Teorema di Church Rosser

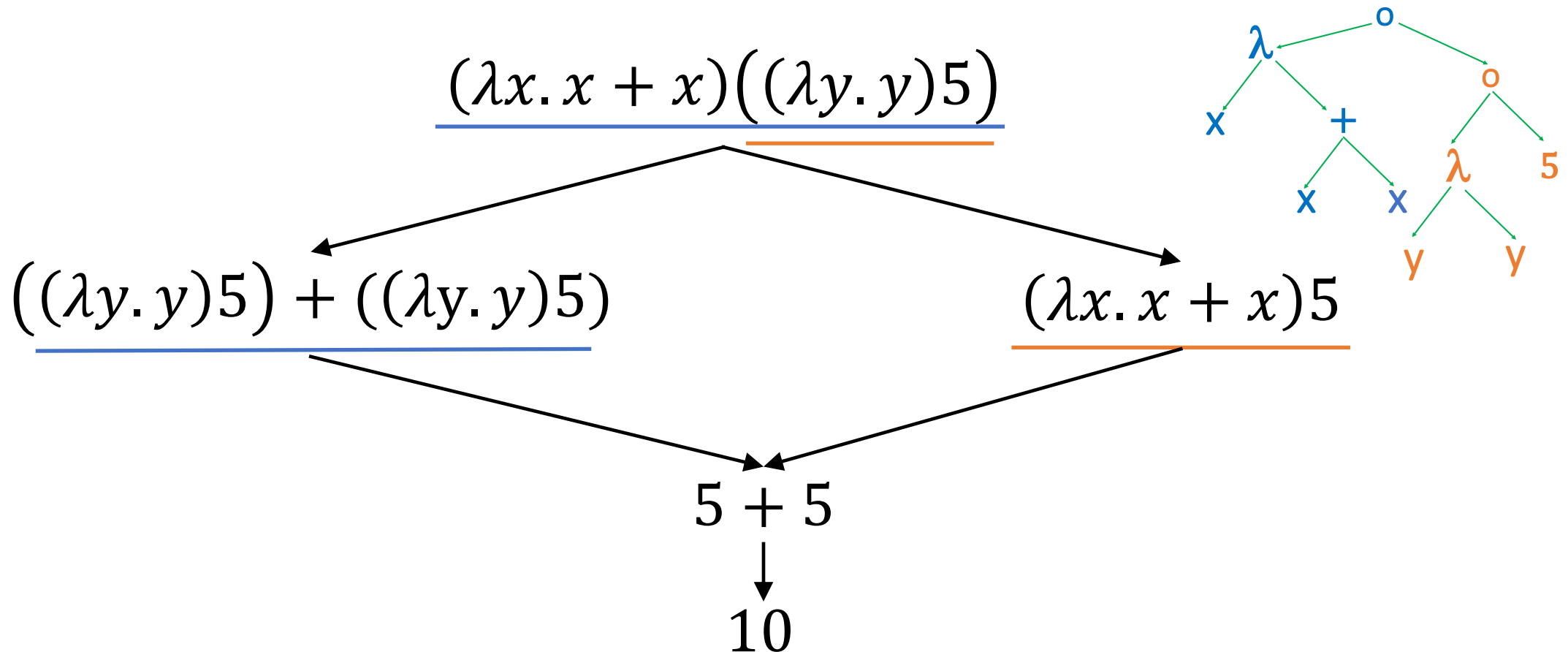
Il **Teorema di Church – Rosser** afferma che l'ordine in cui vengono scelte le beta riduzioni non influisce sul risultato finale

Più precisamente:

- **se** ci sono due riduzioni distinte o sequenze di riduzioni che possono essere applicate alla stessa lambda espressione,
- **allora** esiste una lambda espressione che è raggiungibile da entrambi i risultati, applicando sequenze (possibilmente vuote) di riduzioni aggiuntive

Ordine di riduzione: esempio (cont.)

In caso di più redex, esiste più di un ordine di valutazione, ovvero più di una scelta per applicare le regola di β -riduzione

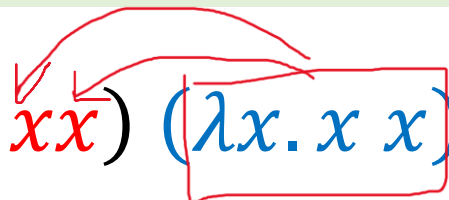


Non terminazione

$$\Omega = (\lambda x. x x)(\lambda x. x x)$$

La lambda espressione Ω **non** può essere ridotta in forma normale
(è un combinatore **divergente**)

Contiene una sola espressione riducibile tramite beta riduzione
La riduzione produce ancora Ω


$$\Omega = (\lambda x. \textcolor{red}{x x}) (\textcolor{blue}{\lambda x. x x}) \rightarrow (\lambda x. x x)(\lambda x. x x) \rightarrow \dots$$

Loop!

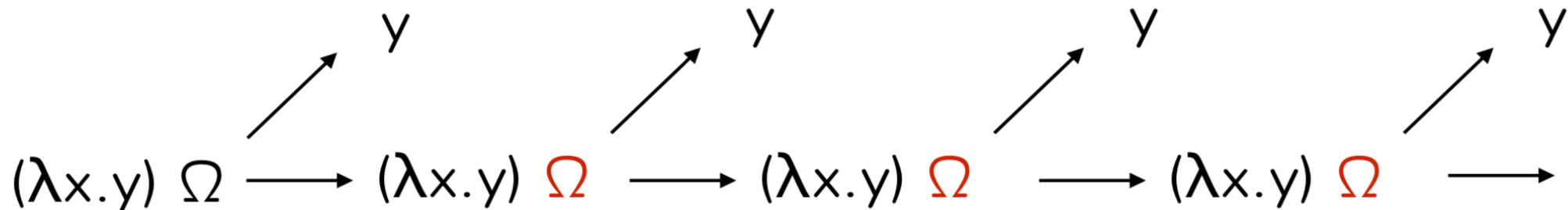
Non terminazione e confluenza

$$\Omega = (\lambda x. x x)(\lambda x. x x)$$

Una lambda espressione può:

- terminare "in alcuni casi" (ossia, operando certe scelte di riduzione)
- e non terminare "in altri casi" (facendo scelte diverse)

Church-Rosser assicura che in tutti i casi in cui la riduzione termina il risultato sarà lo stesso (non possono cioè esserci risultati diversi)



Lambda calcolo e programmazione funzionale

Funzioni di ordine superiore

Il lambda calcolo è più potente di quanto la sua definizione minimale possa suggerire

Non è possibile definire funzioni con più argomenti, tuttavia è facilissimo descrivere il comportamento di funzioni con più argomenti utilizzando funzioni di ordine superiore (higher order functions) ovvero funzioni che producono come risultato del calcolo altre funzioni

Funzioni con più argomenti

- Nel lambda calcolo una funzione ha **esattamente una** variabile legata e può essere applicata a **un solo argomento**
- Le funzioni che prevedono più argomenti devono quindi presentare un annidamento di λ -astrazioni, ovvero prendere un argomento alla volta
- Una funzione che accetta due argomenti si presenta in questo modo:

$\lambda x. (\lambda y. xy)$

- Matematicamente è come passare da una funzione $f: (X \times Y) \rightarrow Z$ a una funzione $g: X \rightarrow (Y \rightarrow Z)$ che calcola gli stessi risultati
- g è una funzione che prende un argomento X e restituisce una funzione che mette in relazione Y e Z
- Questa trasformazione si chiama **currying**

Intermezzo: Haskell Curry

- L'operatore di trasformazione è detto **currying**, in onore del matematico e logico Haskell Curry, che ne studiò a lungo le proprietà
- Nei linguaggi di programmazione il **currying** risulta di fatto **l'applicazione parziale di una funzione ai suoi argomenti**: ogni funzione di più argomenti si considera un funzionale (una funzione che restituisce una funzione) e gli argomenti vengono presi appunto "uno alla volta" a cominciare dal primo dando come risultato altri funzionali o (arrivati al penultimo argomento) una funzione
- Ad esempio, possiamo definire la funzione che prende una funzione (qualsiasi) in ingresso e la applica al secondo parametro due volte

```
Twice = λ f. λ x. f (f x)
Twice (λ y. y + y) → λ x. (λ y. y + y) ((λ y. y + y) x)
Twice (λ y. y + y) 2 → (λ y. y + y) ((λ y. y + y) 2) →→ (λ y. y + y) (4) → 8
```

Funzioni con più parametri formali grazie a ordine superiore

$$F = \lambda(x, y). e$$

$$F(e_1, e_2) \rightarrow e\{x := e_1\}\{y := e_2\}$$

$$F = \lambda x. \lambda y. e$$

L'espressività non cambia

$$(F e_1) e_2 \rightarrow (\lambda y. e)\{x := e_1\} e_2 \rightarrow e\{x := e_1\}\{y := e_2\}$$

Funzione che invocata con il parametro attuale e_1 produce come risultato una funzione che invocata con parametro attuale e_2 produce il risultato richiesto

Nella programmazione

Funzioni con diversi parametri formali

```
function sum(a, b) {  
    return a * b;  
}
```

Utilizziamo la trasformazione, **currying**, che traduce una funzione con diversi parametri in una sequenza di funzioni che prendono ciascuna un singolo argomento

```
res = sum(a, b)
```

```
h = g(a)  
res = h(b)
```

```
res = sum (a) (b)
```

*la funzione **sum** da $\text{Int} \times \text{Int} \rightarrow \text{Int}$ viene trasformata in una funzione **g** che prende in ingresso un intero e restituisce come risultato una funzione da interi a interi $\mathbf{g}: \text{Int} \rightarrow (\text{Int} \rightarrow \text{Int})$*

Currying in Javascript

```
function curry(f) {  
    return function(a) {  
        return function(b) {  
            return f(a, b);  
        };  
    };  
}
```

Notare l'annidamento delle funzioni

```
//uso  
function sum(a, b) { return a + b; }  
  
let curriedSum = curry(sum);  
alert( curriedSum(1)(2) );  
// 3
```

... e la ricorsione?

- Nel lambda calcolo, **non è previsto un meccanismo primitivo** per definire **funzioni ricorsive**
- Possiamo definire funzioni ricorsive nel lambda calcolo?
- Il problema è che le **funzioni sono anonime** e non si possono quindi chiamare ricorsivamente,...
- ... ma possiamo aggirare il problema

Il combinatore di punto fisso Y

Il **combinatore*** **Y** è una **funzione di ordine superiore** usata per implementare la ricorsione in qualsiasi linguaggio di programmazione che non la supporti nativamente

Anche questo combinatore è stato introdotto da Haskell Curry negli anni '40 ed è considerato una delle idee più belle nella programmazione e nella logica

* Un combinatore è un termine chiuso (anche $I = \lambda x. x$ e Ω sono combinatori)

Il combinatore Y

In un contesto dove le funzioni non possono fare esplicitamente riferimento a sé stesse, **Y permette a una funzione di richiamare sé stessa in modo implicito**

Ogni volta che la funzione viene chiamata, non chiama sé stessa esplicitamente, ma un'altra funzione che la richiama al posto suo

$$Y = \lambda f. (\lambda x. f(x\ x))(\lambda x. f(x\ x))$$

Proprietà fondamentale del combinatore Y

$$Y\ F \equiv_{\beta} F(Y\ F)$$

Il combinatore Y

$$Y = \lambda f. (\lambda x. f(x x)) (\lambda x. f(x x))$$

$$\begin{aligned} YF &= \lambda f. (\lambda x. f(x x)) (\lambda x. f(x x)) F \\ &\rightarrow (\lambda x. F(x x)) (\lambda x. F(x x)) \\ &\rightarrow F ((\lambda x. F(x x)) ((\lambda x. F(x x)))) \\ &\rightarrow F \left(F \left((\lambda x. F(x x)) . (\lambda x. F(x x)) \right) \right) \\ &\rightarrow \dots \end{aligned}$$

$(\lambda x. F(x x))$ è passata a sé stessa come argomento, creando una sorta di "copia infinita" della funzione

Il combinatore Y

$$Y = \lambda f. (\lambda x. f(x x)) (\lambda x. f(x x))$$

$$\begin{aligned} YF &= \lambda \textcolor{red}{f}. (\lambda x. f(x x)) (\lambda x. f(x x)) \textcolor{red}{F} \\ &\rightarrow (\lambda x. F(x x)) (\lambda x. F(x x)) \\ &\rightarrow \textcolor{red}{F} ((\lambda x. F(x x)) (\lambda x. F(x x))) \end{aligned}$$

$$\begin{aligned} \textcolor{red}{F} ((\lambda x. F(x x)) (\lambda x. F(x x))) &\leftarrow F(\lambda \textcolor{red}{f}. (\lambda x. f(x x)) (\lambda x. f(x x)) \textcolor{red}{F}) \\ &= F(Y F) \end{aligned}$$

Il combinatore Y

$$Y = \lambda f. (\lambda x. f(x x)) (\lambda x. f(x x))$$

$$\begin{aligned} YF &= \lambda f. (\lambda x. f(x x)) (\lambda x. f(x x)) F \\ &\rightarrow (\lambda x. F(x x)) (\lambda x. F(x x)) \\ &\rightarrow F((\lambda x. F(x x)) (\lambda x. F(x x))) \end{aligned}$$

$$Y F \equiv_{\beta} F(Y F)$$

Dato che $Y F$ e $F(Y F)$
si riducono allo stesso
termine

$$\begin{aligned} F((\lambda x. F(x x)) (\lambda x. F(x x))) &\leftarrow F(\lambda f. (\lambda x. f(x x)) (\lambda x. f(x x)) F) \\ &= F(Y F) \end{aligned}$$

Definizioni ricorsive

Problema: definire una funzione ricorsiva

$F = \langle \text{corpo della funzione contenente } F \rangle$

F rappresenta la logica della funzione ricorsiva, ad esempio, la funzione per il fattoriale

Primo passo:

Definire $G = \lambda f. \langle \text{espressione che contiene } f \rangle$

Secondo passo:

Definire $F = Y G$

$$\begin{aligned}
F &= YG \\
&\equiv_{\beta} G(YG) \\
&\equiv_{\beta} < \textit{espressione con } (Y\ G) > \\
&\equiv_{\beta} < \textit{espressione con } (\textit{espressione con } (Y\ G)) > \\
&\equiv_{\beta} \dots
\end{aligned}$$

Esempio: fattoriale

N.B. supponendo di avere numeri, $IF, n = 0, n - 1, *$

- **$G = \lambda f. \langle \text{espressione che contiene } f \rangle$**

$$G = \lambda f. \lambda n. \text{if } (n = 0) \text{ then } 1 \text{ else } n * f(n - 1)$$

- **Definire $F = Y G$**

$$F = YG = (\lambda f. (\lambda x. f(x x))(\lambda x. f(x x)))G = \\ (\lambda x. G(x x))(\lambda x. G(x x))$$

- Ora abbiamo una funzione che applica G a $(x x)$, dove x è ancora $\lambda x. G(x x)$
- Questo genera una forma ricorsiva, senza fare riferimento esplicito al nome della funzione

Esempio: fattoriale

$$G = \lambda f. \lambda n. \text{if } (n = 0) \text{ then } 1 \text{ else } n * f(n - 1)$$

Applicazione

YG 1 = $(\lambda f. (\lambda x. f(x\ x))(\lambda x. f(x\ x)))G1 \rightarrow ?$

Esempio: fattoriale

$$G = \lambda f. \lambda n. \text{if } (n = 0) \text{ then } 1 \text{ else } n * f(n - 1)$$

Applicazione

$$\begin{aligned} \text{YG } 1 &= (\lambda f. (\lambda x. f(x \ x))(\lambda x. f(x \ x))) G 1 \rightarrow \\ & \quad ((\lambda x. G(x \ x)) (\lambda x. G(x \ x))) 1 \rightarrow \\ & \quad \left(G \left((\lambda x. G(x \ x)) (\lambda x. G(x \ x)) \right) \right) 1 \rightarrow \\ & \quad (\lambda n. \text{if } (n = 0) \text{ then } 1 \text{ else } n * ((\lambda x. G(x \ x)) (\lambda x. G(x \ x))) (n - 1)) 1 \rightarrow \\ & \quad \text{if } (1 = 0) \text{ then } 1 \text{ else } 1 * ((\lambda x. G(x \ x)) (\lambda x. G(x \ x))) (1 - 1) \rightarrow \\ & \quad 1 * ((\lambda x. G(x \ x)) (\lambda x. G(x \ x))) 0 \rightarrow \\ & \quad 1 * \left(G \left((\lambda x. G(x \ x)) (\lambda x. G(x \ x)) \right) \right) 0 \rightarrow \\ & \quad 1 * (\lambda n. \text{if } (n = 0) \text{ then } 1 \text{ else } n * ((\lambda x. G(x \ x)) (\lambda x. G(x \ x))) (n - 1)) 0 \rightarrow \\ & \quad 1 * \text{if } (0 = 0) \text{ then } 1 \text{ else } 0 * ((\lambda x. G(x \ x)) (\lambda x. G(x \ x))) (0 - 1) \rightarrow \\ & \quad 1 * 1 = 1 \end{aligned}$$

Y combinator in Javascript

Auto-applicazione

La funzione f può invocare sé stessa utilizzando questa struttura ricorsiva

```
const Y = f => (x => x(x))(x => f(y => x(x)(y)));

const factorial = f => (x => (x === 1 ? 1 : x * f(x - 1)));

const YFactorial = Y(factorial)(10);
```

La versione Javascript di Y è leggermente diversa, perché qui la valutazione è di tipo **call-by-value** e valutare subito `xx` in `fx` porterebbe a un loop infinito, mentre con `xxy` si ritarda la ricorsione sino a quando `f` viene effettivamente chiamata

Programmare in lambda calcolo

- Il calcolo lambda è molto più potente di quanto la sua minuscola definizione possa far pensare

Il lambda calcolo e i linguaggi di programmazione

- Abbiamo visto come nel lambda calcolo sia possibile definire funzioni ricorsive e funzioni di ordine superiore
- Il lambda calcolo puro tuttavia non ha che funzioni
- È possibile usare booleani, numeri, condizionali (insomma tutto ciò che serve per essere Turing-equivalenti)?
- Si può ricorrere alle codifiche

Come?

- Soffermendosi non su cosa il valore rappresenta ma sul che cosa ci si può fare
- Per ogni tipo di dato pensare a descriverne **l'uso in termini di una funzione**

Costanti

- Mostriamo come sia possibile codificare nel lambda calcolo valori Booleani, numeri naturali e le operazioni significative per operare su tali valori

Booleans

- Cosa si può fare con un booleano? Si può fare una scelta binaria
- Quindi un booleano può essere codificato da una funzione che date due scelte possibili ne seleziona una

Booleans

- Cosa si può fare con un booleano? Si può fare una scelta binaria
- Quindi un booleano può essere codificato da una funzione che date due scelte possibili ne seleziona una

$$\begin{array}{lcl} \textit{TRUE} & = & \lambda t. \lambda f. t \\ \textit{FALSE} & = & \lambda t. \lambda f. f \end{array}$$

- Si possono usare per verificare la verità di un valore booleano
- Si possono codificare facilmente anche gli operatori booleani (NOT, AND, OR,..): come?

Condizionale

- Cosa si può fare con un condizionale? Operare una scelta in base al valore della condizione

$IF = \lambda c. \lambda then. \lambda else. c \text{ then } else$

- restituisce “*then*” se *c* si riduce a *TRUE* e restituisce “*else*” se *c* si riduce a *FALSE* (vedi esempio dopo)

$$IF = \lambda c. \lambda \text{ then. } \lambda \text{ else. } c \text{ then else}$$

$$\begin{aligned}
 IF \text{ TRUE } e_1 e_2 &= (\lambda c. \lambda \text{ then. } \lambda \text{ else. } c \text{ then else}) \text{ TRUE } e_1 e_2 \\
 &\rightarrow (\lambda \text{ then. } \lambda \text{ else. } \text{TRUE then else}) e_1 e_2 \\
 &\rightarrow (\lambda \text{ else. } \text{TRUE } e_1 \text{ else}) e_2 \\
 &\rightarrow \text{TRUE } e_1 e_2 \\
 &= (\lambda t. \lambda f. t) e_1 e_2 \\
 &\rightarrow (\lambda f. e_1) e_2 \\
 &\rightarrow e_1
 \end{aligned}$$

$$\begin{aligned}
 \text{TRUE} &= \lambda t. \lambda f. t \\
 \text{FALSE} &= \lambda t. \lambda f. f
 \end{aligned}$$

Esercizio: Calcolare IF FALSE e₁ e₂

Numeri naturali: Church numerals

- Come si possono codificare i numeri naturali? Esprimendo i numeri come operatori attivi
- Possiamo codificare un numero naturale con una funzione che, data una funzione *s* (la funzione *iteratore del successore*) e un valore iniziale *z* (il *costruttore zero*), applica *s* un numero di volte pari al numero *n* che si vuole codificare: come se *n* fosse un iteratore

INTUIZIONE: il **numero 3** significa semplicemente fare qualcosa (applicare una funzione, come il successore) per **tre volte**

$$C_0 = \lambda s. \lambda z. z$$

$$C_1 = \lambda s. \lambda z. s z$$

$$C_2 = \lambda s. \lambda z. s(s z)$$

...

$$C_n = \lambda s. \lambda z. s(s(...(s z)...) = \lambda s. \lambda z. s^n z$$

Operazioni sui numeri naturali: iszero

- Si possono codificare (con un po' di pazienza) anche le operazioni sui numeri naturali: successore, addizione, prodotto,...

ISZERO =?

$$C_0 = \lambda s. \lambda z. z$$

$$C_1 = \lambda s. \lambda z. sz$$

Esercizio:

- Definire *ISZERO* in modo che $ISZERO\ C_0 = TRUE$ e $ISZERO\ C_i = FALSE$ per $i \neq 0$
- Calcolare $ISZERO\ C_1$

Operazioni sui numeri naturali: successore

- Si possono codificare (con un po' di pazienza) anche le operazioni sui numeri naturali: successore, addizione, prodotto,...

$$SUCC = \lambda n. \lambda s. \lambda z. s(nsz)$$

- dove n è l'operando (numerale di Church), z è il valore iniziale, e s è il successore: in nsz applica s a z per n volte, al risultato viene applicata s , quindi complessivamente s viene applicata a z per $n + 1$ volte

- **Esercizio:** Calcolare $SUCC C_1$

$$\begin{aligned} C_1 &= \lambda s. \lambda z. sz \\ C_2 &= \lambda s. \lambda z. s(sz) \end{aligned}$$

Operazioni sui numeri naturali: addizione

$$\begin{aligned} SUCC &= \lambda n. \lambda s. \lambda z. s(nsz) \\ PLUS &= \lambda m. \lambda n. \lambda s. \lambda z. m \ s(n \ s \ z) = \\ &\quad \lambda m. \lambda n. m \ SUCC \ n \end{aligned}$$

- dove m ed n sono i due addendi, z è il valore iniziale, e s è la funzione successore
- (nsz) calcola n , applicando la funzione successore a z n volte
- $ms(nsz)$ applica m al risultato di (nzs) cioè applica la funzione successore ad n per m volte: quindi porta al risultato
- **Esercizio:** Calcolare $PLUS \ C_1 \ C_1 = \lambda m. \lambda n. m \ SUCC \ n \ C_1 \ C_1 \rightarrow \rightarrow$

Operazioni sui numeri naturali: addizione

$$SUCC = \lambda n. \lambda s. \lambda z. s(nsz)$$

$$PLUS = \lambda m. \lambda n. \lambda s. \lambda z. ms \ (n \ s \ z) = \\ \lambda m. \lambda n. m \ SUCC \ n$$

$$TIMES = \lambda m. \lambda n. m \ (PLUS \ n) \ C_0$$

- In *TIMES* il numero *m* rappresenta quante volte sommare *n*. La somma viene fatta con *PLUS*
- *m* è visto come un contatore di iterazioni: ogni iterazione applica *PLUS n* al risultato corrente (partendo da *C₀*, cioè 0)

Passaggio dei parametri

Passaggio dei parametri

- Qual è la regola per la **valutazione dell'applicazione di funzione**?
- **Primo passo:** valutazione dell'espressione che definisce la funzione
- **Secondo passo:** definizione del meccanismo di passaggio dei parametri. Ci sono due alternative per sostituire le occorrenze della variabile legata nell'espressione del corpo della funzione:
 1. **[valutazione eager – call-by-value (CBT)]** le occorrenze sono sostituite dal **valore dell'espressione** del parametro formale
 2. **[valutazione lazy – call-by-name (CBN)]** le occorrenze sono sostituite dall'**espressione non valutata** che definisce il parametro attuale
- Dopo il passaggio dei parametri, si passa a valutare il corpo della funzione
- Haskell adotta lo stile lazy, mentre OCaml quello eager (così come la maggior parte dei linguaggi)

La nozione di riduzione - definizione standard, senza strategie di valutazione (non deterministica)

La **β riduzione**, $\rightarrow$, è la relazione definita dalle regole seguenti

$$(\lambda x. e_1) e_2 \rightarrow e_1 \{x := e_2\}$$

$$\frac{e_1 \rightarrow e'}{e_1 e_2 \rightarrow e' e_2}$$

$$\frac{e_2 \rightarrow e'}{e_1 e_2 \rightarrow e_1 e'}$$

$$\frac{e \rightarrow e'}{\lambda x. e \rightarrow \lambda x. e'}$$

Qualsiasi redex può essere ridotto in qualsiasi momento

Riduzione Call-By-Value (CBV)

$$(\lambda x. e_1) v \rightarrow e_1 \{x := v\}$$

$$\frac{e_1 \rightarrow e'}{e_1 e_2 \rightarrow e' e_2}$$

$$\frac{e_2 \rightarrow e'}{v e_2 \rightarrow v e'}$$

Prima si valuta completamente l'argomento e poi si applica la funzione (NOTA: non scende nel corpo delle funzioni... come nei linguaggi di programmazione)

Riduzione Call-By-Name (CBN)

$$(\lambda x. e_1) e_2 \rightarrow e_1 \{x := e_2\}$$

$$\frac{e_1 \rightarrow e'}{e_1 e_2 \rightarrow e' e_2}$$

Non si valuta l'argomento prima della chiamata. Si applica cioè la funzione il prima possibile (ma portandola prima in forma $\lambda x. e_1$) (NOTA: non si scende nel corpo delle funzioni... come nei linguaggi di programmazione)

Call by Value: esempio

$((\lambda x. (\lambda y. (yx))) (5 + 2)) (\lambda x. x + 1)$

$(\lambda x. \lambda y. y x) (5 + 2) \lambda x. x + 1$

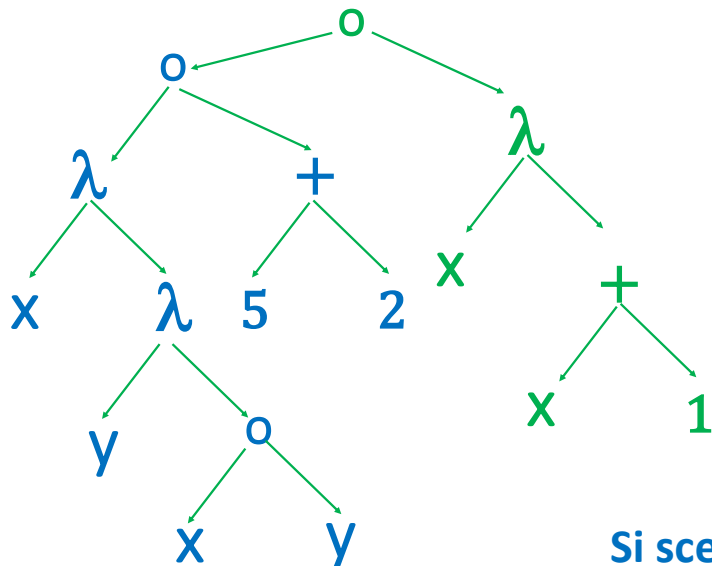
$\longrightarrow (\lambda x. \lambda y. y x) 7 \lambda x. x + 1$

$\longrightarrow (\lambda y. y 7) \lambda x. x + 1$

$\longrightarrow (\lambda x. x + 1) 7$

$\longrightarrow 7 + 1$

$\longrightarrow 8$



Si sceglie il redex più interno

Call by Name: esempio

$((\lambda x. (\lambda y. (yx))) (5 + 2)) (\lambda x. x + 1)$

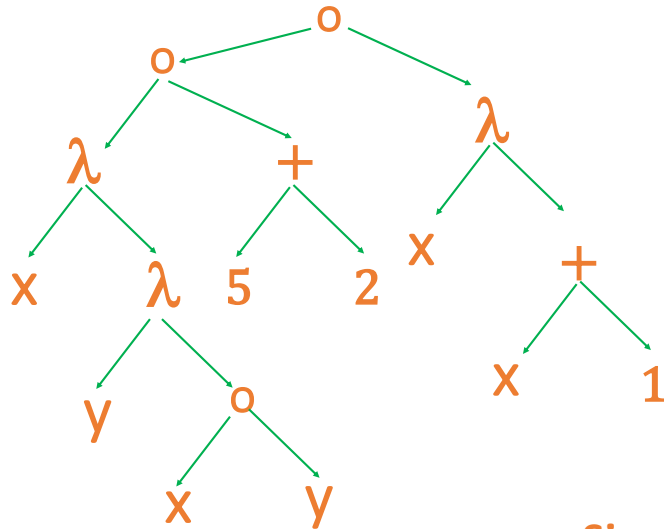
$(\lambda x. \lambda y. y x) (5 + 2) \lambda x. x + 1 \longrightarrow (\lambda y. y (5 + 2)) \lambda x. x + 1$

$\longrightarrow (\lambda x. x + 1) (5 + 2)$

$\longrightarrow (5 + 2) + 1$

$\longrightarrow 7 + 1$

$\longrightarrow 8$



Si sceglie il redex più esterno

Discussione

Supponendo che la valutazione di una lambda espressione termini sempre non importa quale ordine di valutazione si sceglie: si ottiene la stessa risposta

Al contrario, in caso di non terminazione, il comportamento osservabile differisce tra call-by-value e call-by-name

Call by Value vs Call by Name

Analizziamo il comportamento di

IF True True (Ω Ω)

Call by Name

IF True True (Ω Ω) $\Rightarrow$ True

TERMINAZIONE

Call by value

IF True True (Ω Ω) $\Rightarrow$

IF True True (Ω Ω) $\Rightarrow$

IF True True (Ω Ω) $\Rightarrow$

IF True True (Ω Ω) $\Rightarrow$...

NON TERMINAZIONE

Attenzione! Differenze rispetto riduzione standard (senza strategie)

$$\frac{e \rightarrow e'}{\lambda x. e \rightarrow \lambda x. e'}$$

- Le strategie call-by-name e call-by-value **non** hanno la **regola di inferenza che consente di ridurre il corpo di una funzione prima di applicarla**
- L'assenza di questa regola rende queste strategie **non equivalenti** alla riduzione standard
- Ad esempio, l'espressione $\lambda x. ((\lambda y. y)z)$ si può ridurre in $\lambda x. z$ applicando la riduzione standard, mentre non si può ridurre (rimane così com'è) adottando una delle due strategie
- Questa differenza sostanziale è però in linea con ciò che accade nei linguaggi di programmazione: **il corpo di una funzione non viene eseguito fintanto che la funzione non viene invocata!**

Definizioni per il λ -calcolo (non tipato)

Sintassi	$e ::= x \mid \lambda x.e \mid e \ e$
Variabili libere	$FV(x) = \{x\}$ $FV(e_1 \ e_2) = FV(e_1) \cup FV(e_2)$ $FV(\lambda x.e) = FV(e) \setminus \{x\}$
Capture avoiding substitution	$x\{x := e\} = e$ $y\{x := e\} = y \text{ se } x \neq y$ $e_1 e_2 \{x := e\} = (e_1 \{x := e\})(e_2 \{x := e\})$ $(\lambda y.e_1)\{x := e\} = \lambda y.(e_1 \{x := e\})$ se $y \neq x$ e $y \notin FV(e)$ $(\lambda y.e_1)\{x := e\} = \lambda z.((e_1 \{y := z\})\{x := e\})$ se $y \neq x$, $y \in FV(e)$ e z fresca $(\lambda x.e_1)\{x := e\} = \lambda x.e_1$
β-riduzione	$\frac{e_1 \rightarrow e'}{e_1 e_2 \rightarrow e' e_2} \quad \frac{e_2 \rightarrow e'}{e_1 e_2 \rightarrow e_1 e'} \quad \frac{e \rightarrow e'}{\lambda x.e \rightarrow \lambda x.e'}$
Call-by-value	$(\lambda x.e_1)v \rightarrow e_1\{x := v\}$
Call-by-name	$(\lambda x.e_1)e_2 \rightarrow e_1\{x := e_2\}$
	$\frac{e_1 \rightarrow e'}{e_1 e_2 \rightarrow e' e_2} \quad \frac{e_2 \rightarrow e'}{v e_2 \rightarrow v e'} \quad \frac{e_1 \rightarrow e'}{e_1 e_2 \rightarrow e' e_2}$
β-equivalenza	$e_1 \equiv_\beta e_2$ se vale una tra:
	$e_1 \equiv_\alpha e_2 \quad e_1 \Rightarrow e_2 \quad e_2 \Rightarrow e_1 \quad e_1 \Rightarrow e \text{ e } e_2 \Rightarrow e$
Codifiche	$True = \lambda t.\lambda f.t$ $False = \lambda t.\lambda f.f$ $IF = \lambda c.\lambda \text{ then}.\lambda \text{ else}.c \text{ then else}$ $C_0 = \lambda s.\lambda z.z$ $C_1 = \lambda s.\lambda z.sz \quad \dots$ $C_n = \lambda s.\lambda z.s(\dots s(sz))$ $Succ = \lambda n.\lambda s.\lambda z.n(nsz)$ $Plus = \lambda m.\lambda n.\lambda s.\lambda z.m(nzs)s = \lambda m.\lambda n.m \text{ Succ } n$ $Times = \lambda m.\lambda n.m \text{ (Plus } n) C_0$

Discussione



Il lambda calcolo è Turing completo

Può rappresentare praticamente qualsiasi
costrutto dei linguaggi di programmazione.
Utilizzando codifiche intelligenti



Ma i programmi sarebbero

Piuttosto lenti (migliaia di chiamate a funzioni)
Centinaia di righe di codice)
Difficile da capire



In pratica: usiamo linguaggi più ricchi che includono già le
opportune primitive linguistiche

La necessità di avere tipi

Nel lambda calcolo tutti i valori sono codificati tramite le funzioni

- $\text{True} = \lambda t. \lambda f. t =_{\alpha} \lambda x. \lambda y. x$
- $\text{Zero} = \lambda z. \lambda s. z =_{\alpha} \lambda x. \lambda y. x$

Pertanto possiamo facilmente usare valori nei contesti sbagliati

- $\text{IF } 0 \text{ e1 e2}$
- $\text{True } 0$

La stessa cosa accade nei linguaggi di basso livello (assembler)

- Una istruzione è una parola (di macchina): un mucchio di bit
- Tutte le operazioni operano su parole

Prossimo passo:

λ calcolo tipico